Bias-variance-noise decomposition Gradient boosting

Vladislav Goncharenko
ML researcher

girafe ai

MIPT, spring 2025

Recap

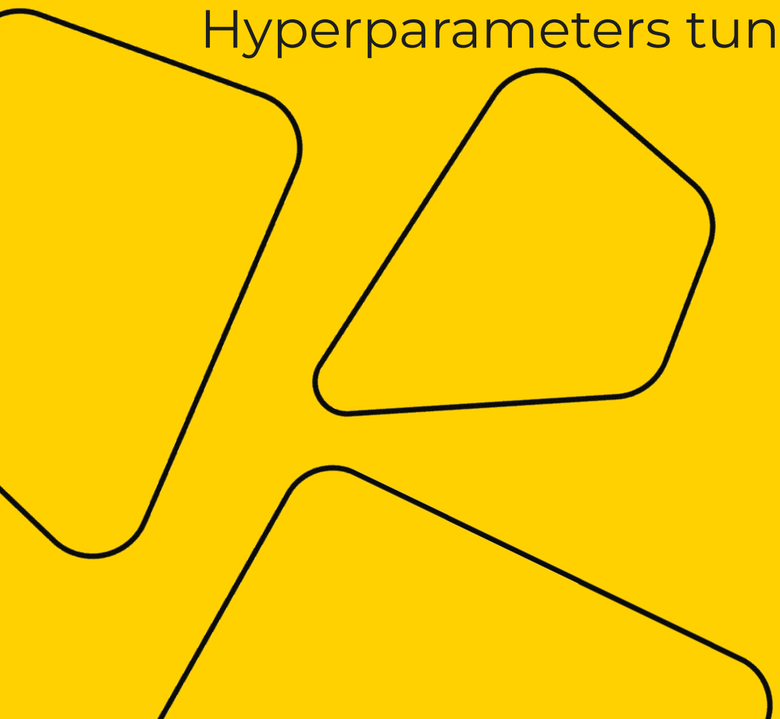
Lecture 5:
Decision trees
Bagging



1. Decision trees intuition
2. Construction procedure
3. Information criteria
4. Special highlights
    a. Dealing with missing data
    b. Binarization
    c. Decision tree as linear model
    d. Standards
    e. Hyperparameters
5. Bootstrap and Bagging
    a. Random Forest

Outline

Lecture 6:
BVN decomposition
Gradient Boosting
Feature importances
Hyperparameters tuning



1. Bias-Variance-Noise decomposition
    a. Bias-Variance tradeoff
    b. Beyond MSE
    c. Double descent
2. Gradient boosting
    a. Intuition
    b. Boosting as gradient descent
    c. Basic model choice
    d. Parallelism
3. Libraries
    a. XGboost, LightGBM, Catboost
4. Feature importances
    a. SHAP
5. Hyperparameter optimization
    a. Optuna

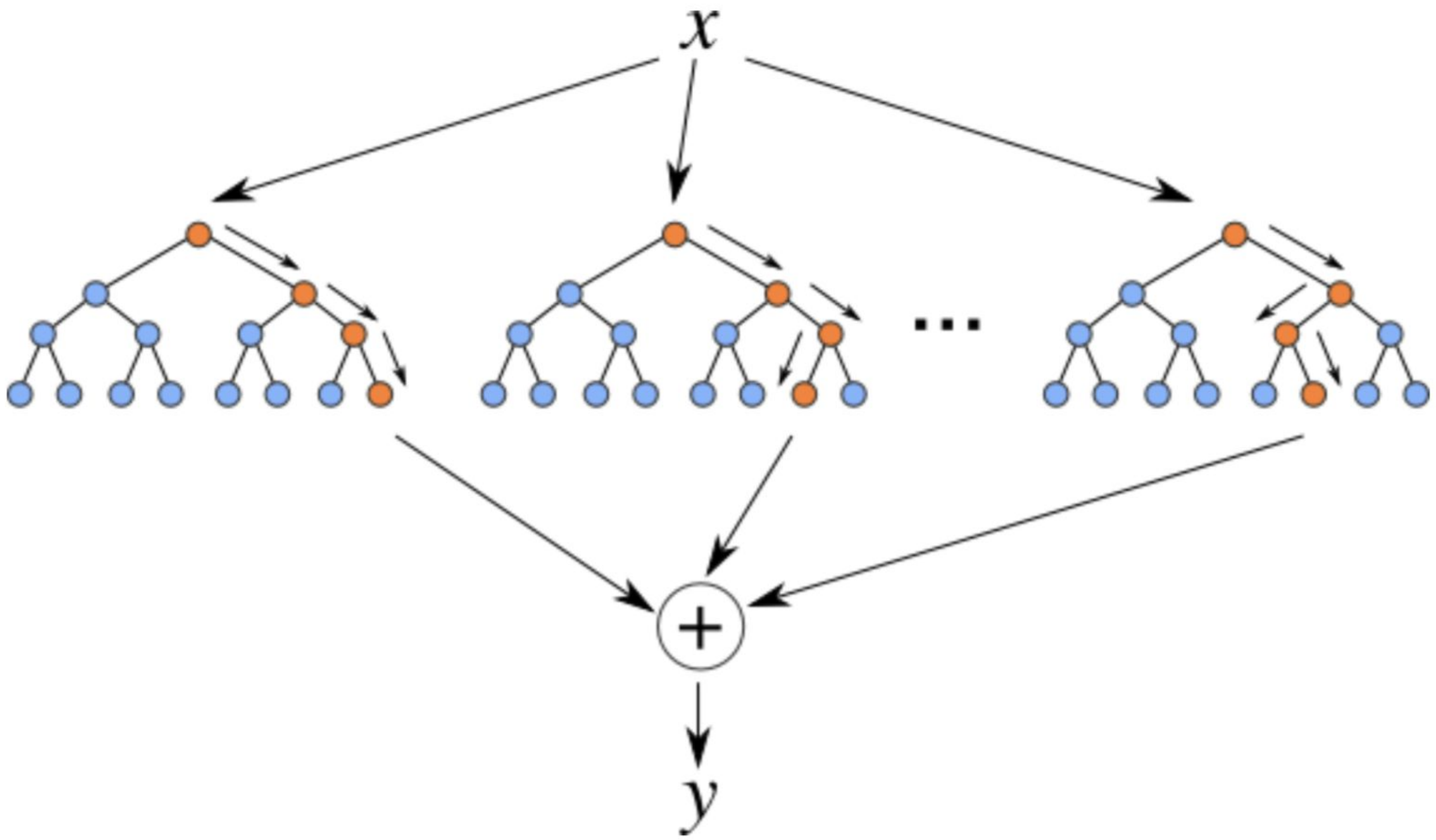
Ensembling recap

girafe ai

00

Random Forest

Bagging + RSM = Random Forest



Random Forest

- One of the greatest “universal” models
- There are some modifications: Extremely Randomized Trees, Isolation Forest, etc.

Bias-variance-noise decomposition

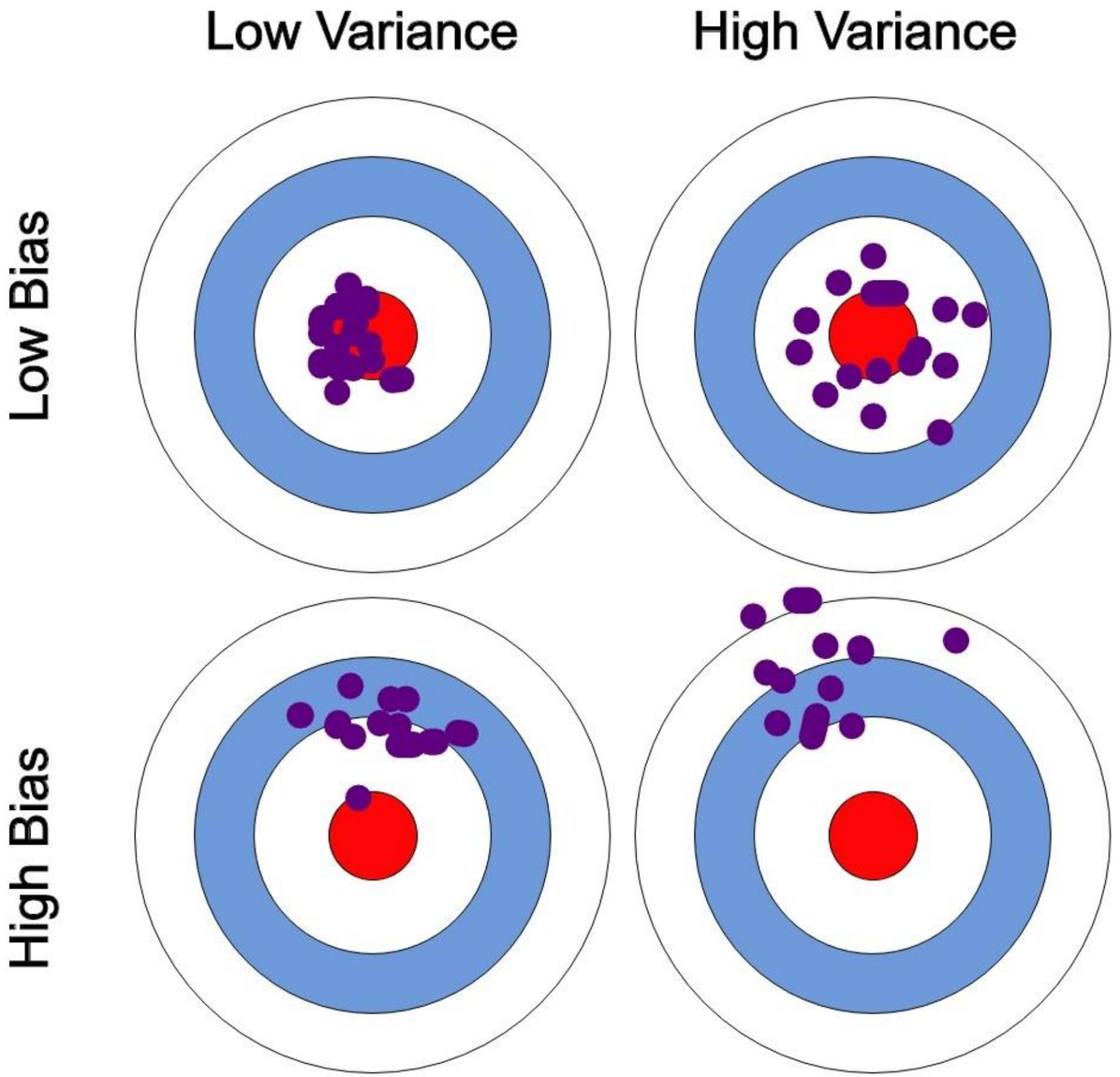
girafe ai

01

Recap

1. Regression model
    a. $y = f(x) + e$
    b. $f(x) \sim E(Y); e \sim Var(Y)$
2. $Var(Y) = ?$
    a. $E(Y^2) - (E(Y))^2$

Intuition



Error is a distance to the center (it is scalar)

Randomness in error



1. Random sampling in training procedure
    a. for Linear model - initial weights
    b. for Tress - feature and threshold sampling
    c. for SGD - sampling of batches
2. Computational errors due to limited precision arithmetics
3. Target noise
    a. epsilon in regression model
4. Objects sampling
    a. Train set sampling
    b. Validation set sampling
5. Humans 🙈

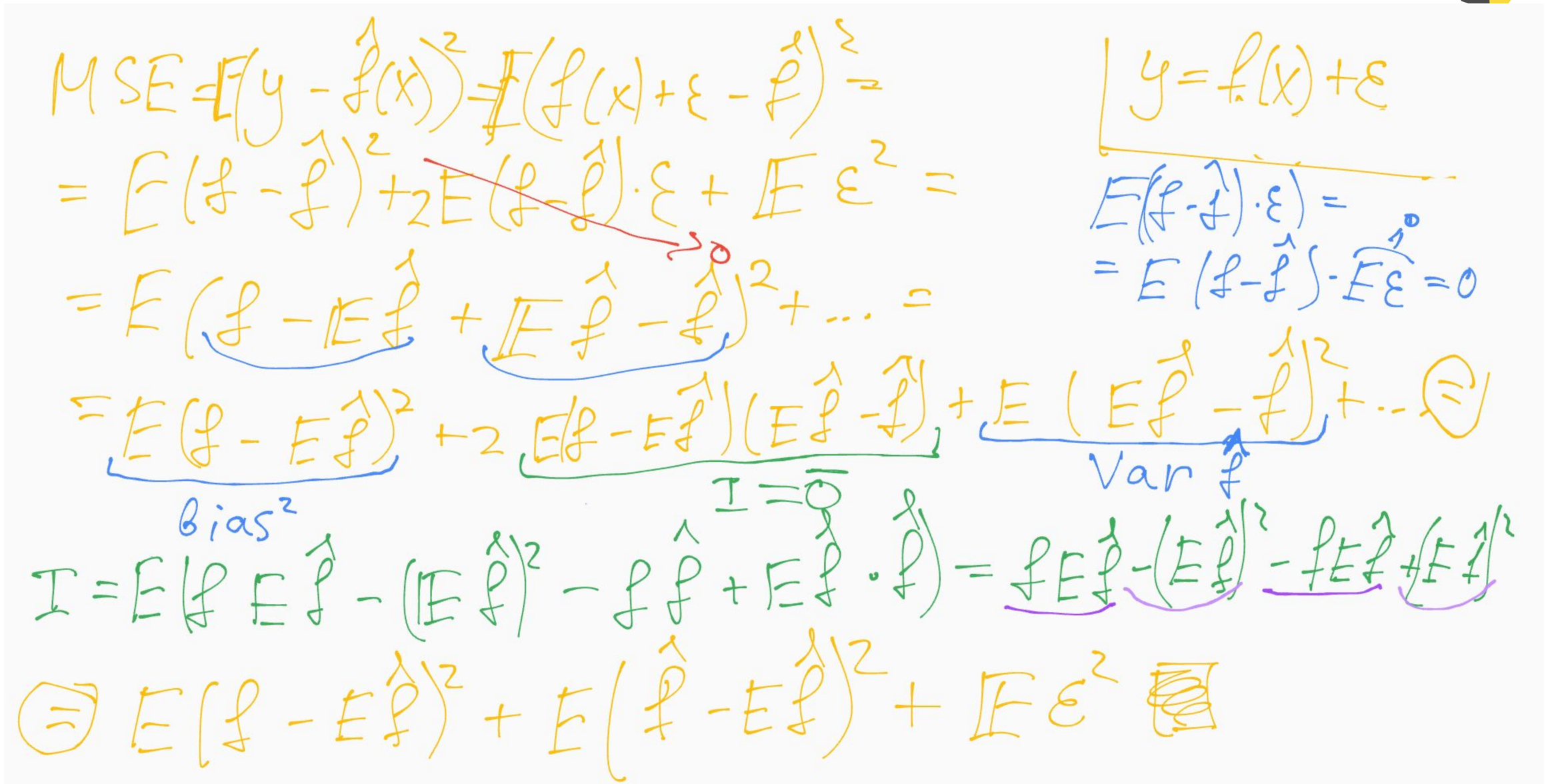
Bias-variance-noise decomposition



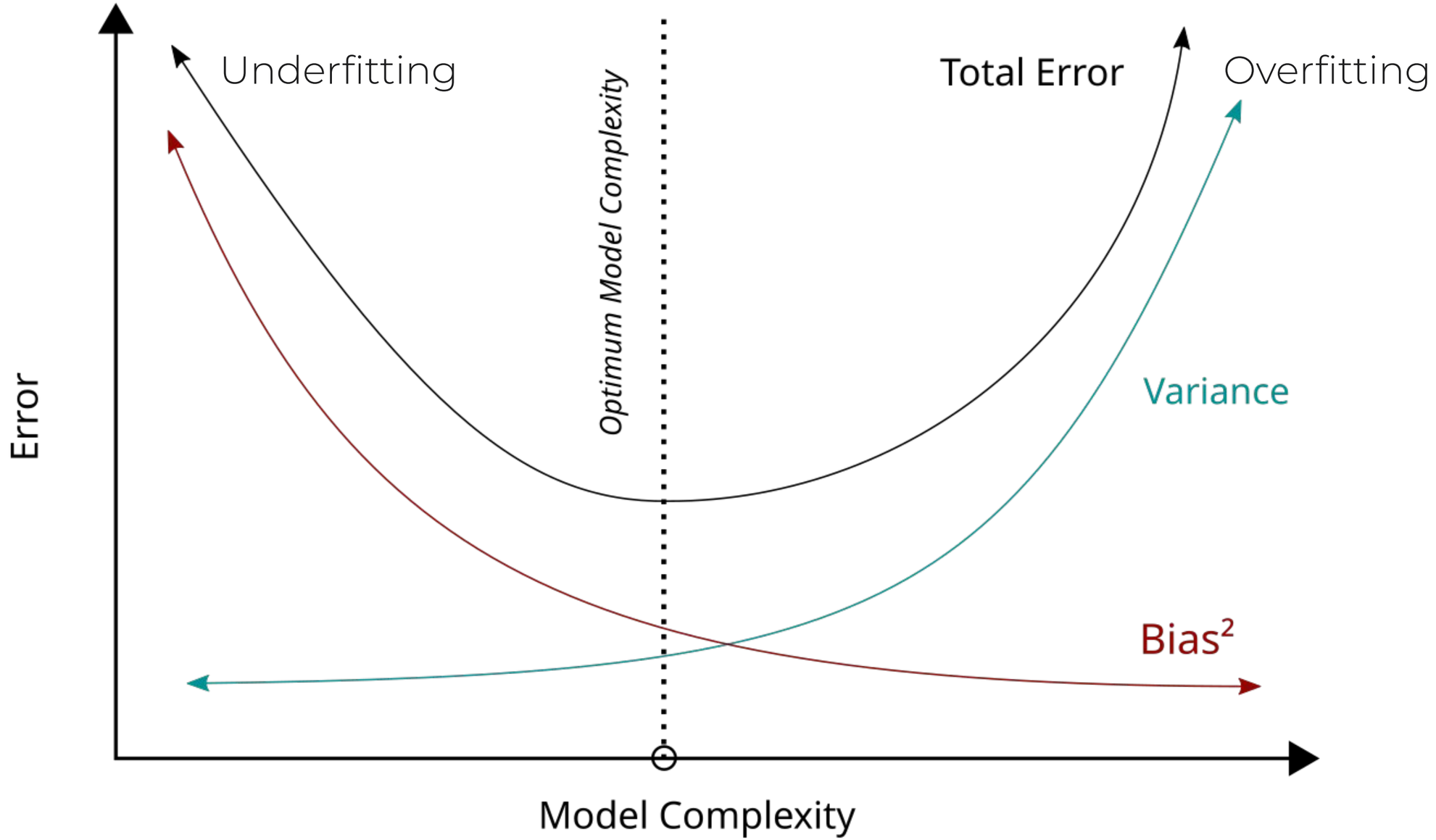
$$\begin{aligned}\mathrm{MSE} &= \left(f(x) - \mathbb{E}\left[\hat{f}\,(x)\right]\right)^2 + \mathbb{E}\left[\left(\mathbb{E}\left[\hat{f}\,(x)\right] - \hat{f}\,(x)\right)^2\right] + \sigma^2 \\ &= \mathrm{Bias}\left(\hat{f}\,(x)\right)^2 + \mathrm{Var}\left[\hat{f}\,(x)\right] + \sigma^2\end{aligned}$$

https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff

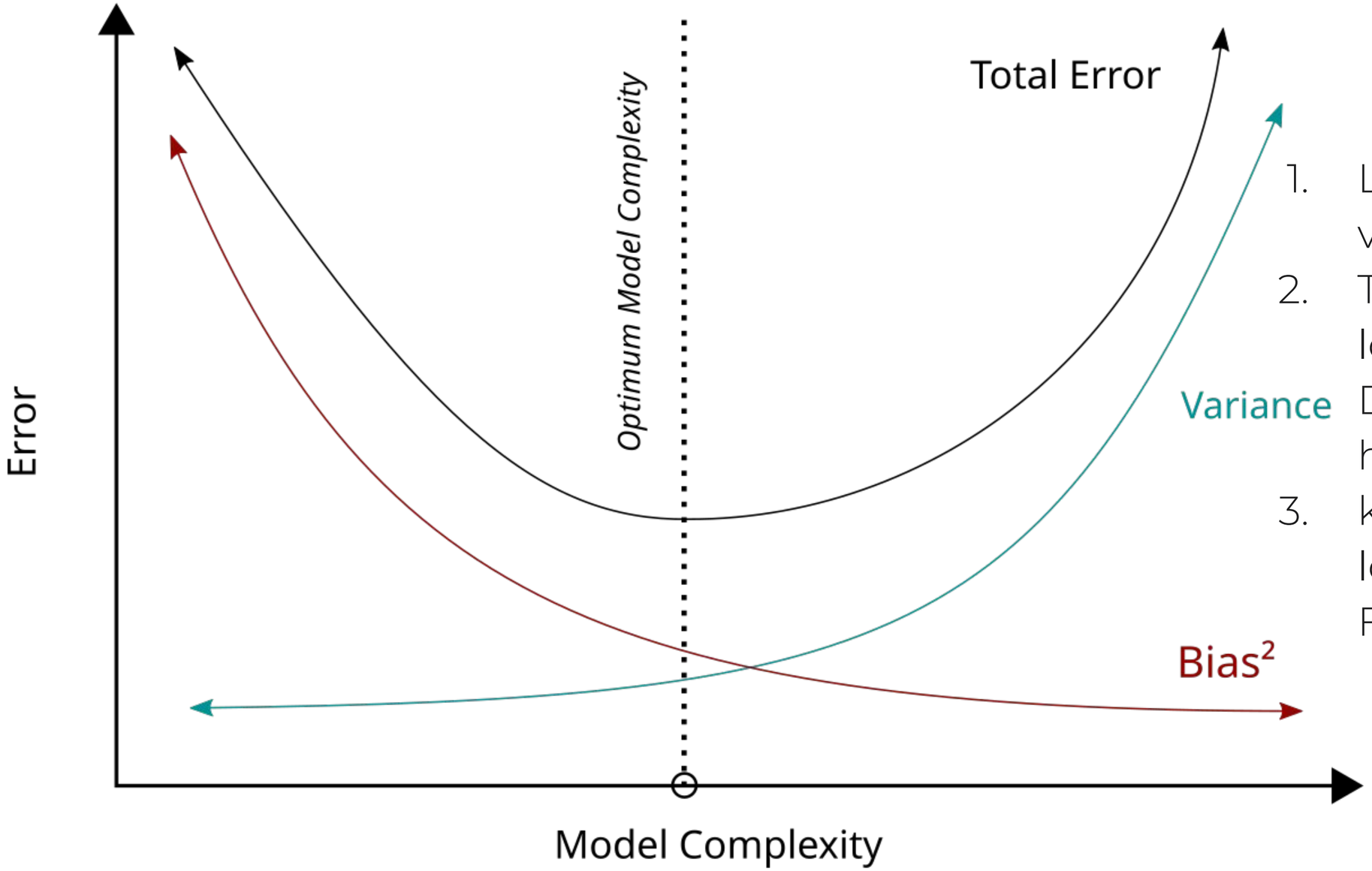
Bias-variance-noise decomposition



Bias-variance tradeoff



Place existing models on axis



1. Linear models - low variance, high bias
2. Trees - high variance, low bias for big trees. Depends on hyperparameters
3. kNN- high variance, low bias.
   For low k values.

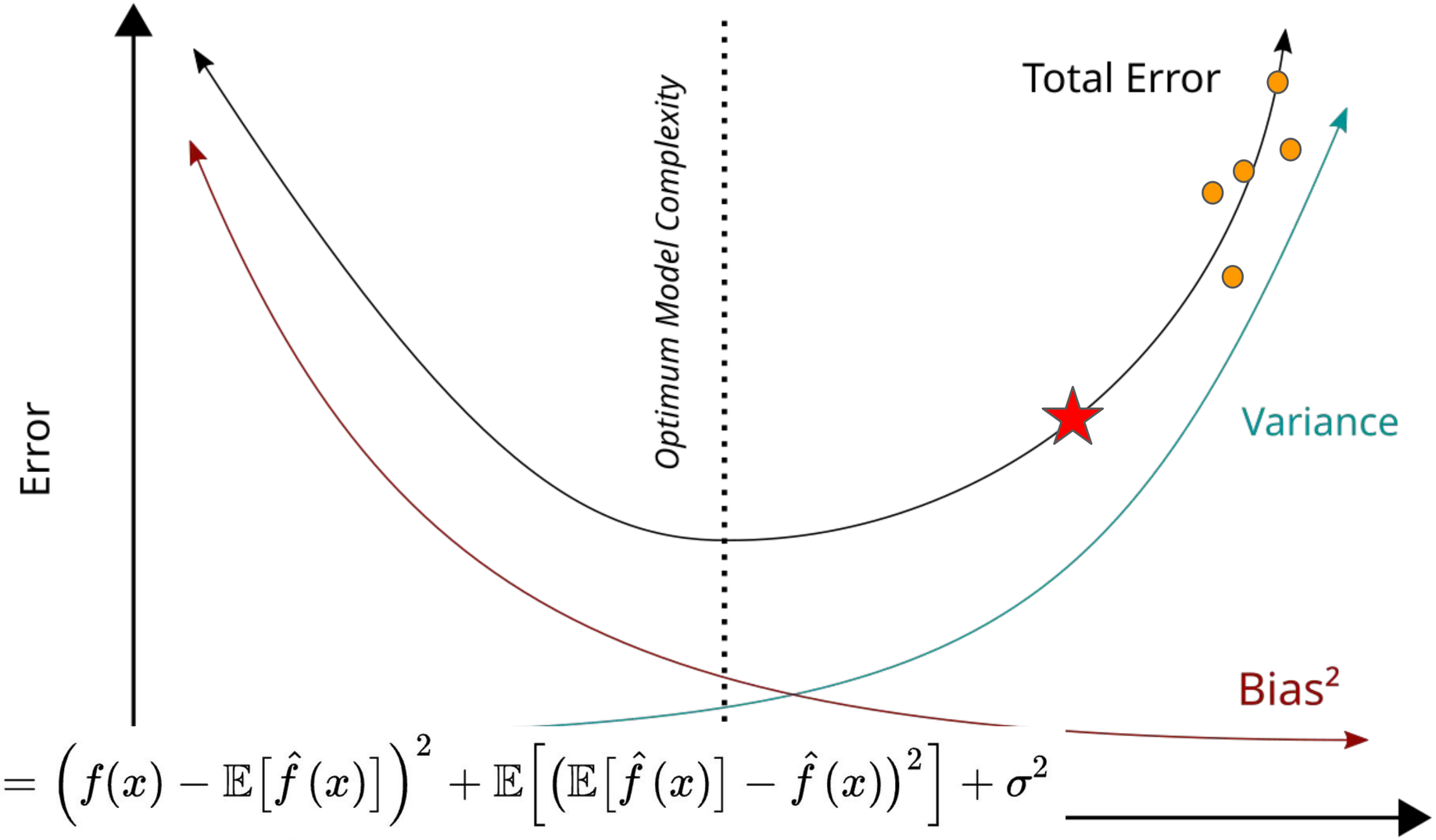
Bias and Variance intuition



1. Matrix inversion has stability score
2. Data resampling in trees

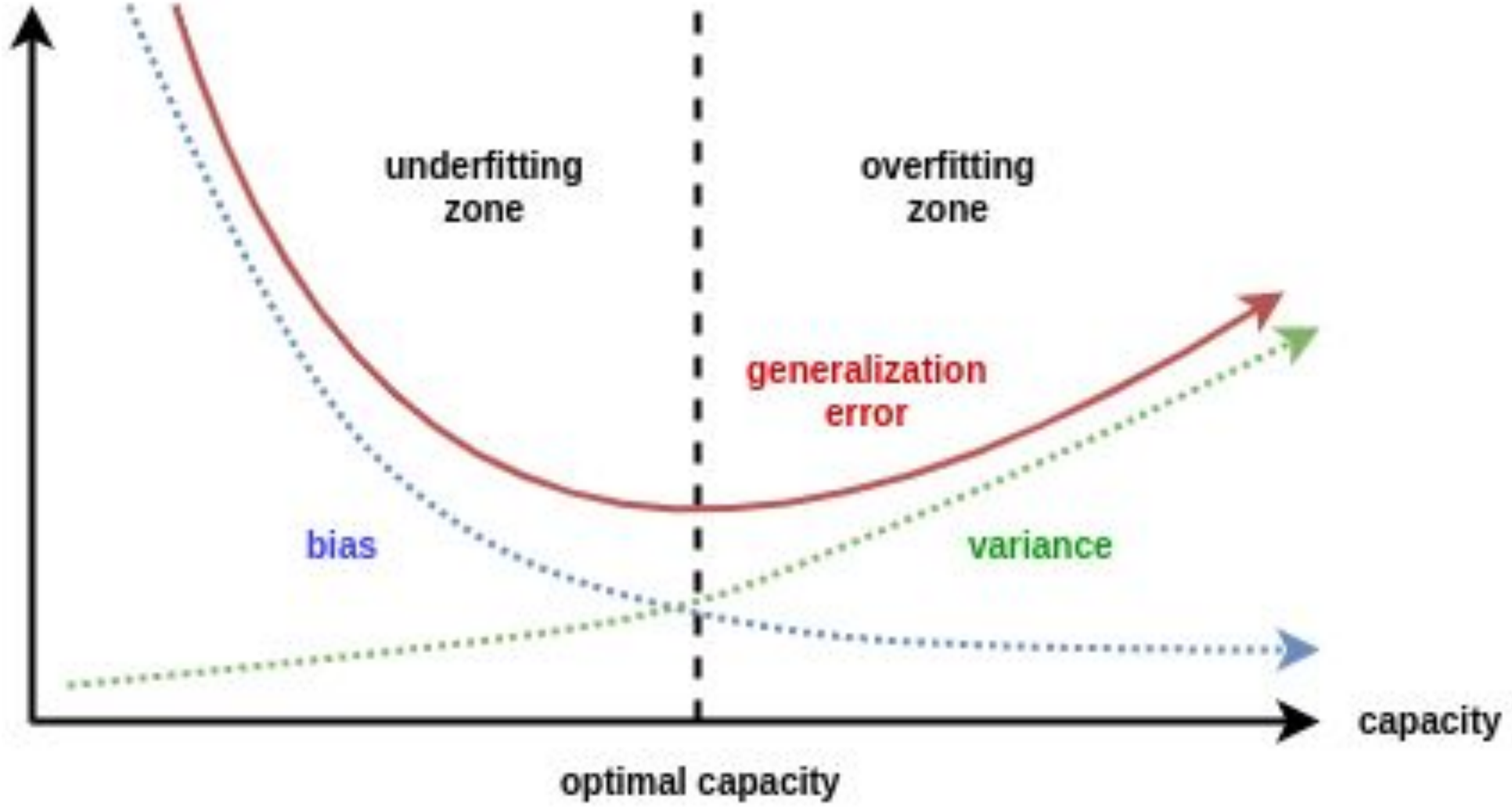
Variance is usually suppressed by big data sample (e.g. for neural networks)

Bagging motivation



$$\begin{aligned}\text{MSE} &= \left(f(x) - \mathbb{E}\left[\hat{f}\,(x)\right]\right)^2 + \mathbb{E}\left[\left(\mathbb{E}\left[\hat{f}\,(x)\right] - \hat{f}\,(x)\right)^2\right] + \sigma^2 \\ &= \text{Bias}\left(\hat{f}\,(x)\right)^2 + \text{Var}\left[\hat{f}\,(x)\right] + \sigma^2\end{aligned}$$

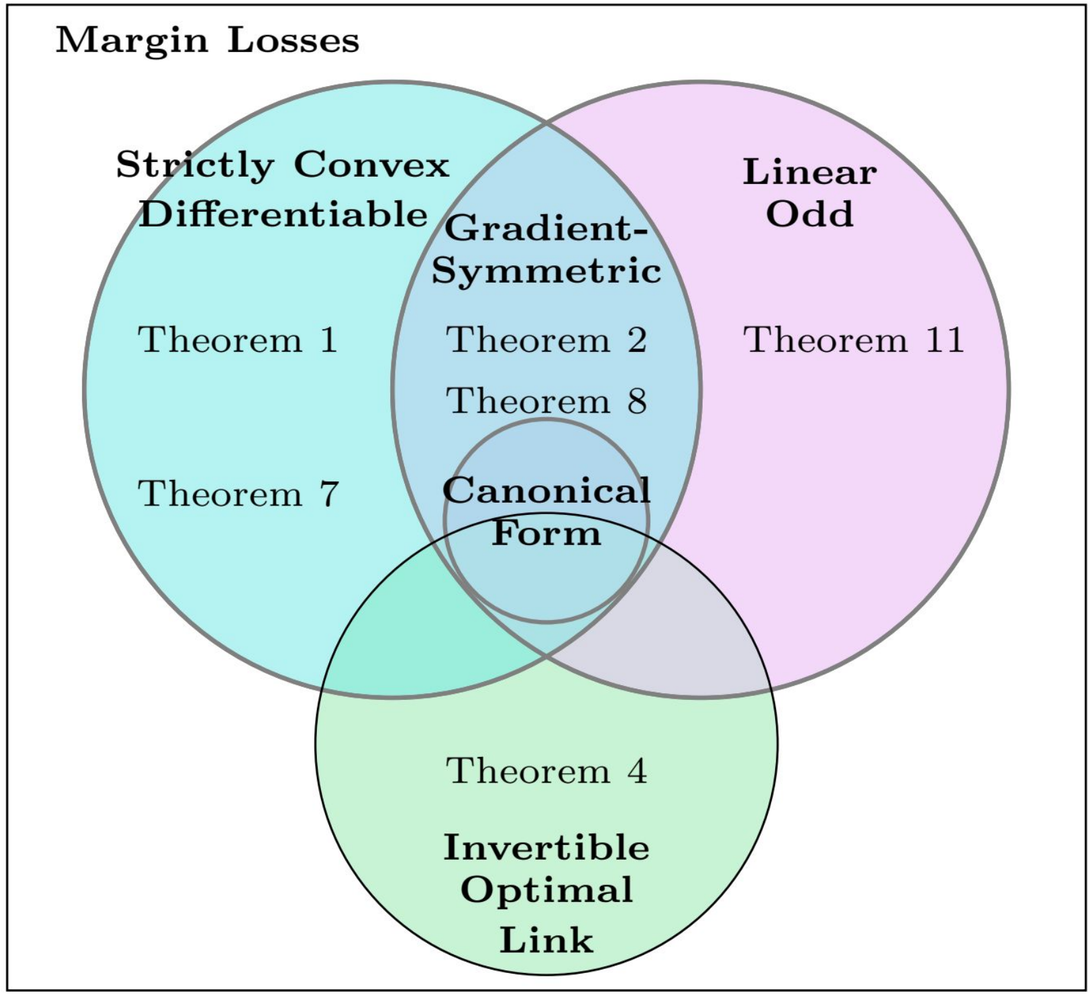
Bias-variance tradeoff



Beyond MSE

Expected Risk =
Risk-of-the-“central”-model + Variation-term

1. A Unified Bias-Variance Decomposition and its Applications, Domingos, 2000 https://homes.cs.washington.edu/~pedrod/papers/mlc00a.pdf
2. Bias-Variance Decompositions for Margin Losses, Wood, et al, 2022, PMLR https://proceedings.mlr.press/v151/wood22a.html

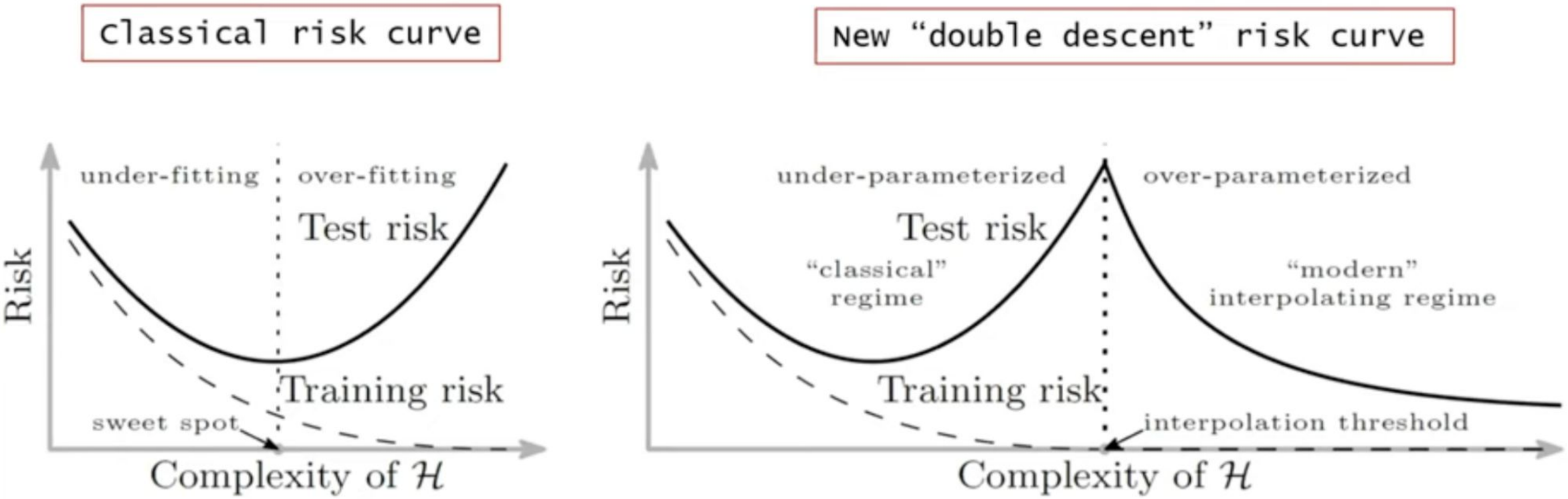


More info



1. https://cnguyen10.github.io/posts/bias-variance-decomposition/
2. https://michelia-zhx.github.io/posts/2022-03-03-bias-variance-decomposition/
3.

Contemporary hypothesis



For contemporary LLMs and other big models "double descent" theory is being developed, see https://www.youtube.com/watch?v=waJOSLNhHII or https://www.youtube.com/watch?v=5-QjjOYfeSI
https://theoreticalecology.wordpress.com/2023/07/18/the-end-of-the-bias-varia

Implementations



1. https://rasbt.github.io/mlxtend/user_guide/evaluate/bias_variance_decomp/
2.

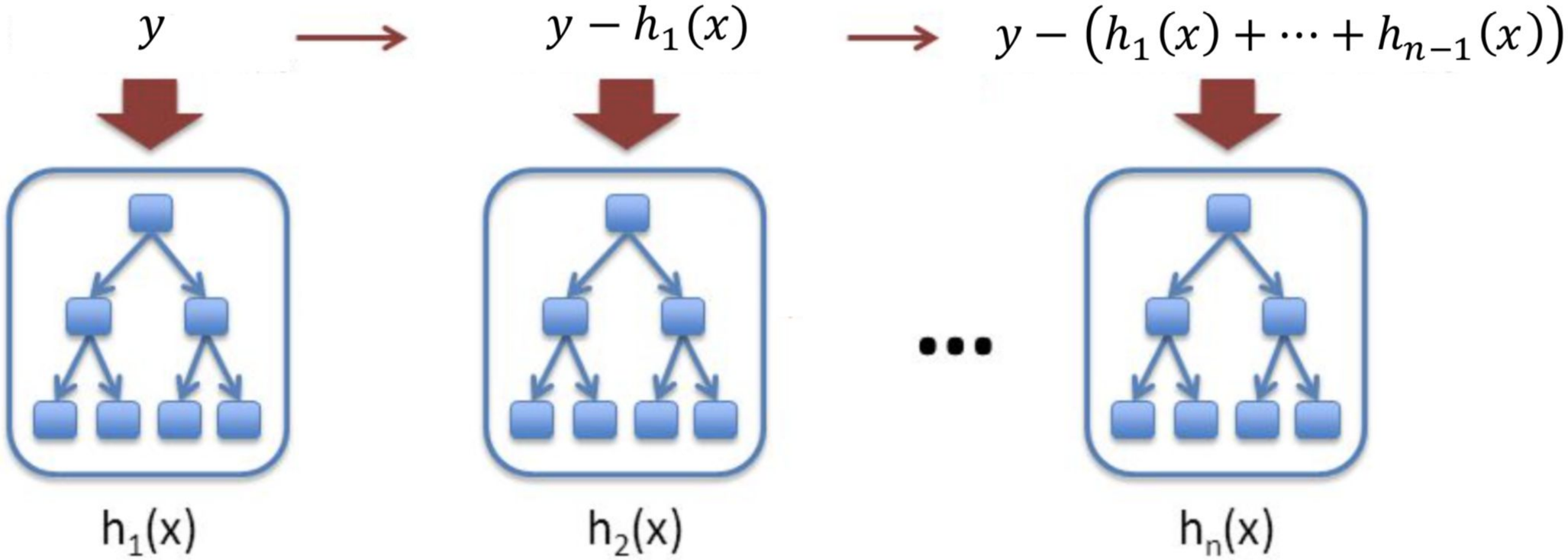
Boosting intuition

girafe ai

02

Boosting for _MSE_

$$a_n(x) = h_1(x) + \cdots + h_n(x)$$

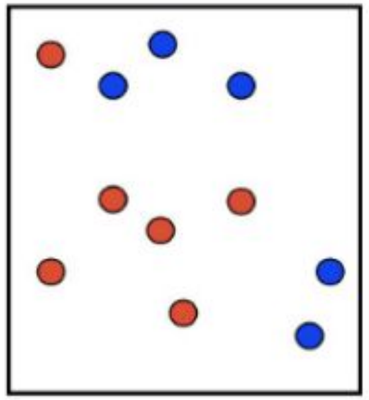


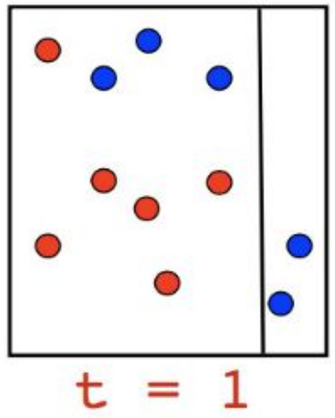
*** only in case of MSE loss!!!**

Boosting: intuition

Binary classification

Use decision stumps

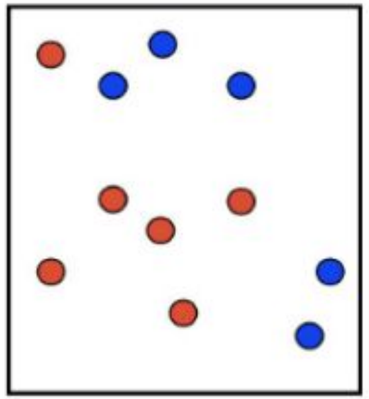


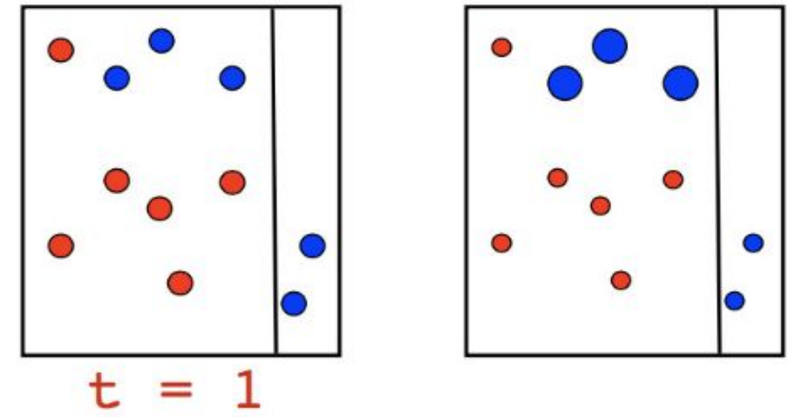


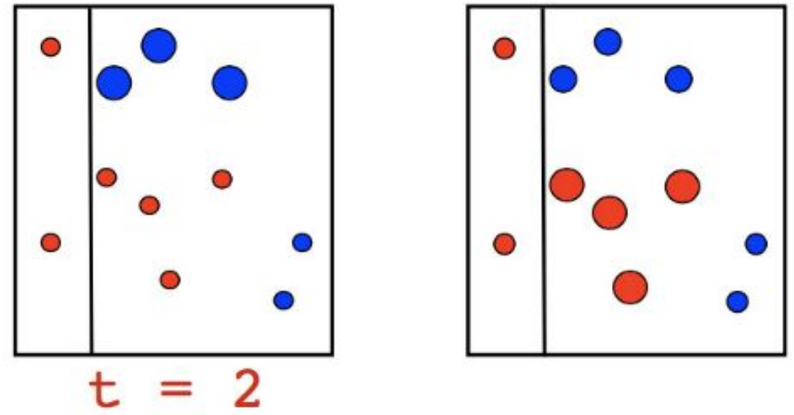
Boosting: intuition

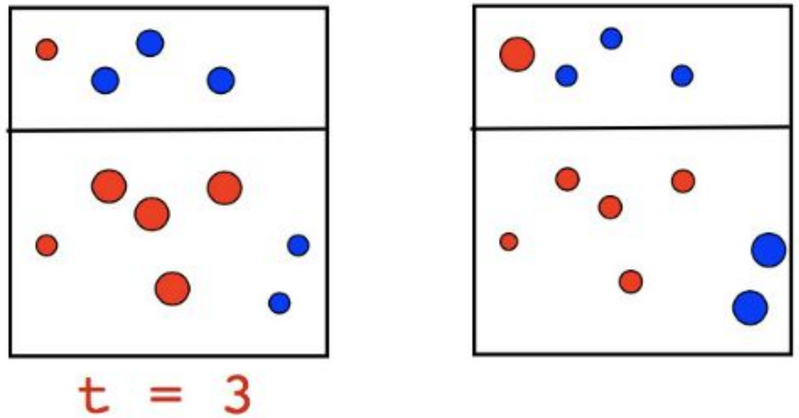
Binary classification

Use decision stumps.





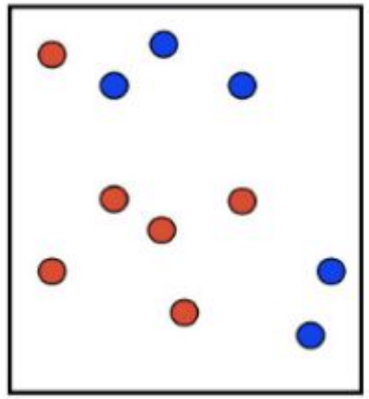


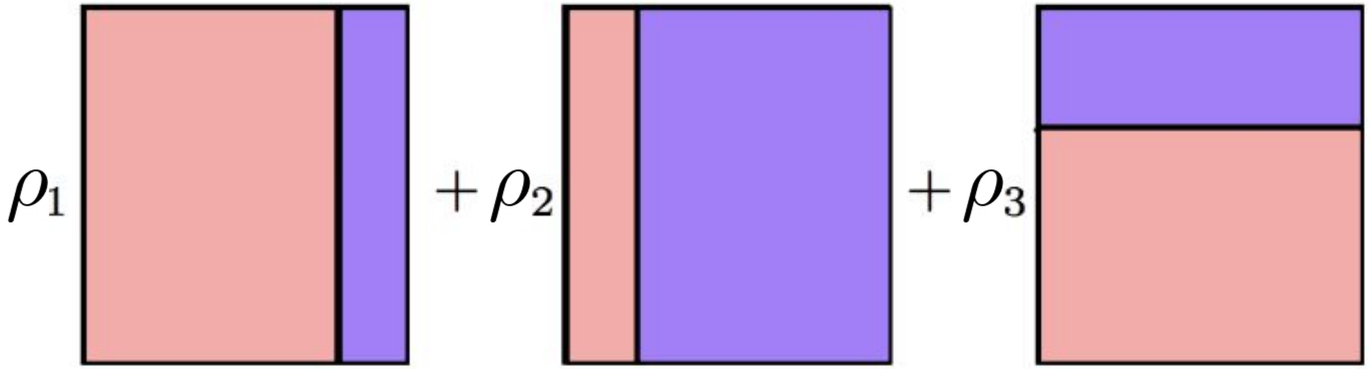


Boosting: intuition

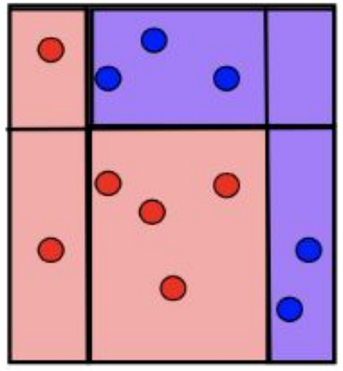
Binary classification

Use decision stumps.

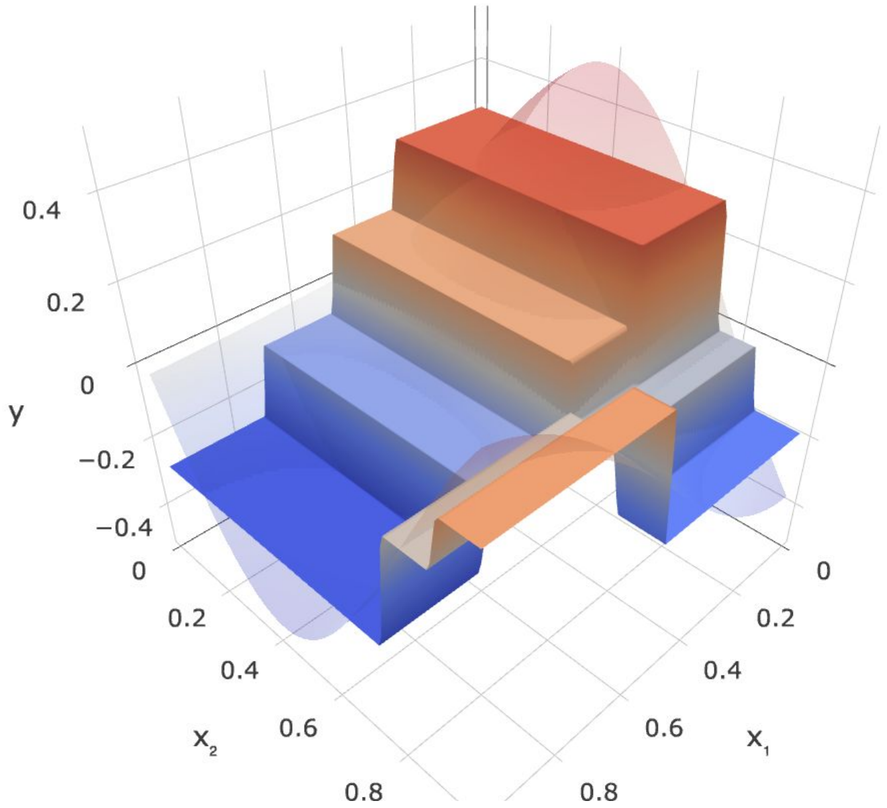




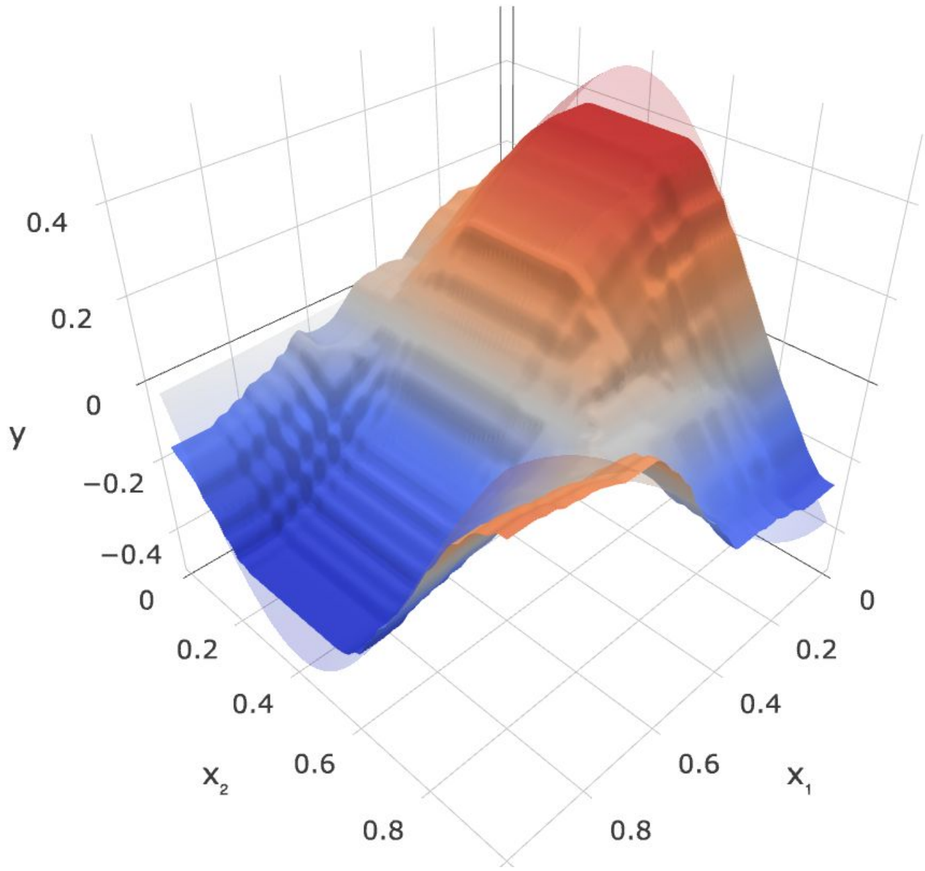
$$\hat{f}_T(x) = \sum_{t=1}^{T} \rho_t h_t(x) \quad =$$



Tree vs GB demo



One tree



Boosting

http://arogozhnikov.github.io/2016/06/24/gradient_boosting_explained.html

Gradient boosting theory

girafe ai

02

Model as vector of predictions

Let's assume we trained first model to predict original target, then we only care about this model's predictions on training dataset.

$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in R^{n \times p} \qquad Y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \bar{y}^* \in R^n$$

$$f_1 = fit(X, Y)$$

$$\bar{f}_1 = f_1(X) = \begin{pmatrix} f_1(x_1) \\ f_1(x_2) \\ \vdots \\ f_1(x_n) \end{pmatrix}$$

How good is this model?

$$L(\bar{y}^*, \bar{f}_1) = L_{f_1}$$

Ensemble as vector of predictions

The same notation can be applied to an ensemble.

$$g_K(x) = \sum_{i=1}^{K} f_i(x)$$

$$\bar{g}_K = \begin{pmatrix} g_K(x_1) \\ g_K(x_2) \\ \vdots \\ g_K(x_n) \end{pmatrix}$$

How to extend ensemble?

$$\min_{\bar{g}_K} L(\bar{y}, \bar{g}_K)$$

Ideally $\bar{y} = \bar{g}_K$

Ensemble extension method

We want to extend ensemble additively, so let's put that idea to formulas:

$$L(\bar{y}^*, \bar{g}_k) = L(\bar{y}^*, \bar{g}_{k-1} + \bar{f}_k) \to \min_{\bar{f}_k}$$

Existing ensemble is fixed and we want to vary only the last term.

Usually L and y* are complex, so we can't directly find such f_k that will minimize loss globally. But we can find f_k which will make situation a bit better than it was.

A small step towards y* and minimal L value.
Which can be found by linearization of the expression with respect to f_k.

Optimal local extension

Local linearization leads to taking first derivative:

$$\frac{\partial L(\bar{y}^*, \bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} = \frac{\partial L(\bar{y}^*, \bar{q})}{\partial \bar{q}} \cdot \frac{\partial (\bar{g}_{k-1} + \bar{f}_k)}{\partial \bar{f}_k} =$$

$$= \frac{\partial L(\bar{y}^*, \bar{p})}{\partial \bar{p}}\Bigg|_{\bar{p} = \bar{g}_{k-1}} \equiv \partial L(\bar{g}_{k-1})$$

p here means whole ensemble prediction

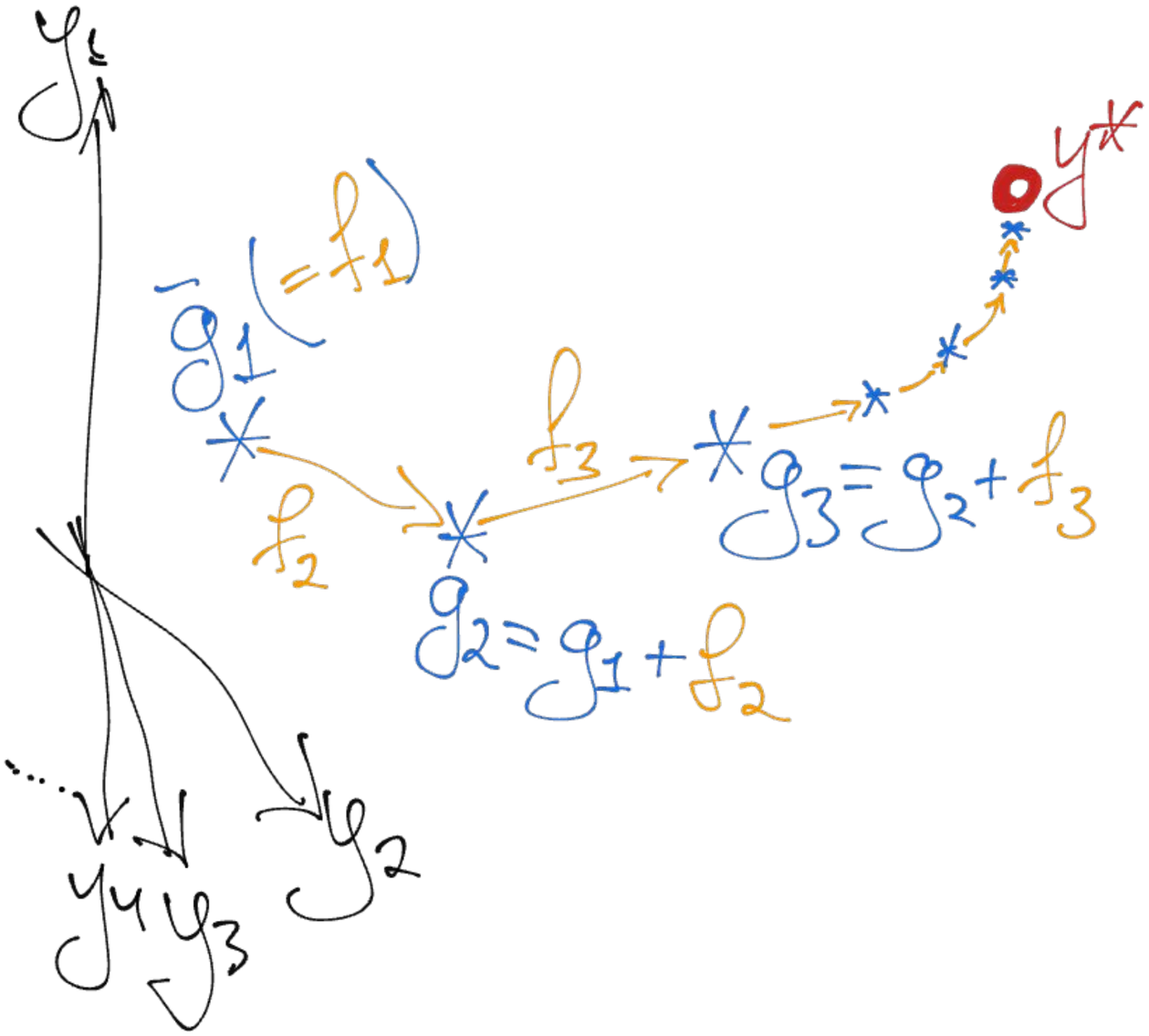
dL is a vector!

$$\bar{g}_k = \bar{g}_{k-1} - lr \cdot \partial L(\bar{g}_{k-1})$$

$= f_k$

Gradient Descent formula

Geometrical interpretation



Vector space for all **f**, **g** and **y*** is a space where each coordinate *i* corresponds to an answer on the object *x_i* in training dataset.

And we are doing regular Gradient Descent in that space.

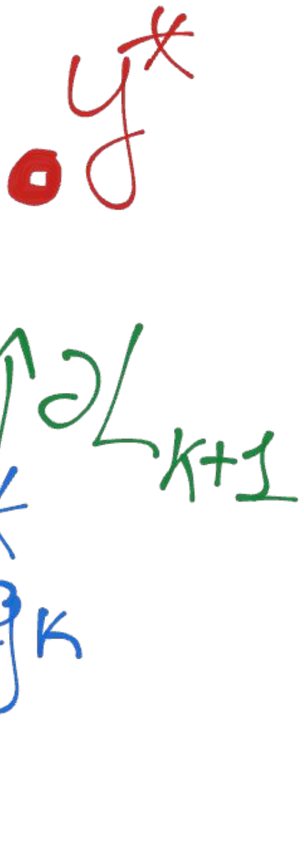
Next model predicts the best step towards right answer.

Basic model choice

Why not linear? Why not kNN? or others?

Why trees as basic algorithms?

1. GB doesn't need differentiable base model
   a. target is given independently between iterations
2. Trees have high variance
   a. gradient varies a lot between iterations
   b. meaning high variance in target for basic model
3. Error on any step doesn't ruin whole algorithm
   a. we recompute true gradients on each step
   b. meta-algorithm (GB) fixes such errors!!!



So for now trees are the only option for GB

Important loss functions

Two most frequently used losses are MSE and Crossentropy

$$L_{MSE}(\bar{y}, \bar{p}) = \frac{1}{n} \sum_{j=1}^{n} (y_i - p_i)^2$$

$$\frac{\partial L_{MSE}}{\partial p_i} = \frac{1}{n} 2(p_i - y_i)$$

$$L_{CE}(\bar{y}, \bar{p}) = \frac{1}{n} \sum_{j=1}^{n} \sum_{k=1}^{c} [y_i = k] \cdot \log p_i^k$$

$$\frac{\partial L_{CE}}{\partial p_i} = ???$$

Other boostings



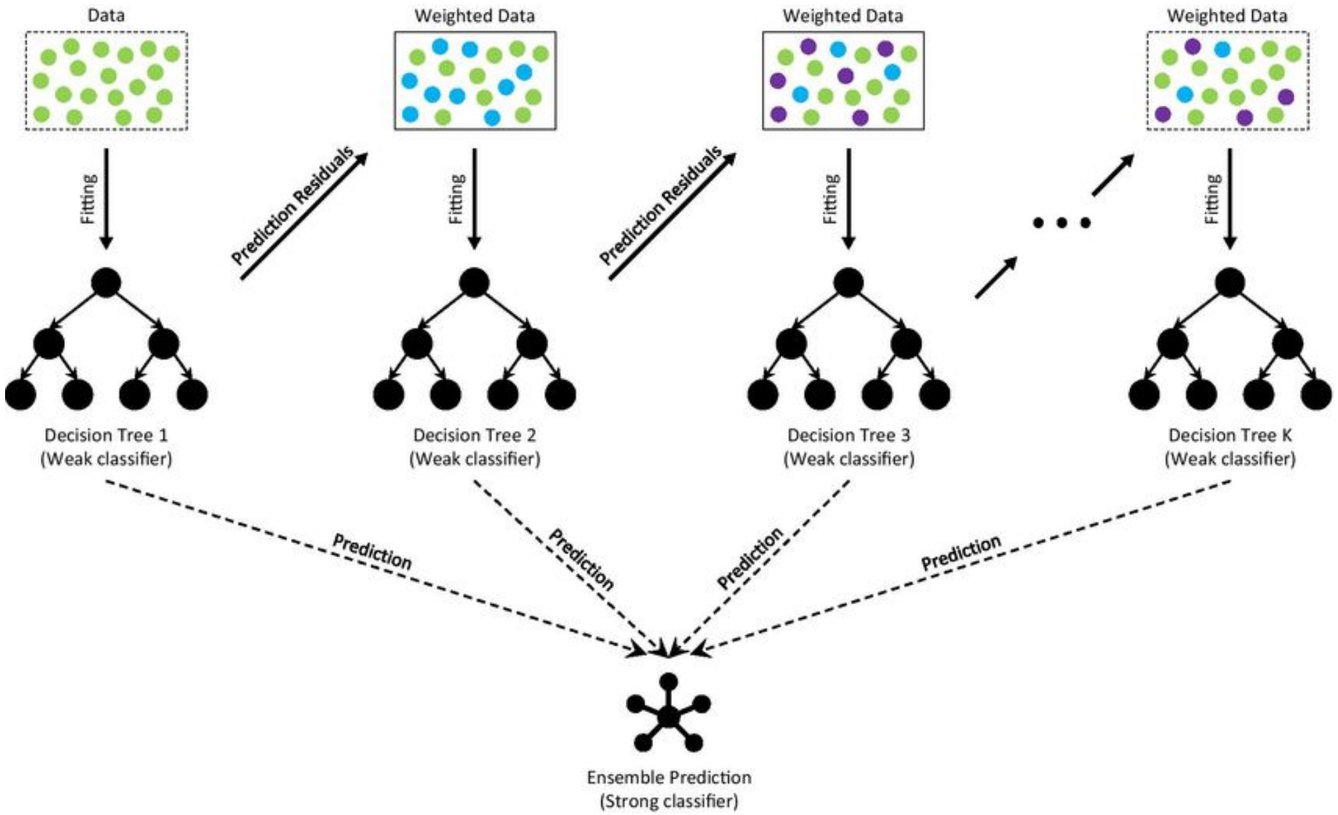
Historically there were a lot of boosting algorithms
(where each next model fixes previous ensemble).
Here is a list of selected algorithms:

- AdaBoost [1995]
- LogitBoost
- BrownBoost
- CoBoosting
- LPBoost
- TotalBoost

As of today Gradient Boosting overperformed them all.

So one more **Bitter lesson** for us

Parallelization



Unlike Bagging, training for GB is done by one tree and can not be parallelized.

But for inference you can run all GB trees in parallel and then sum up results.

	Training	Inference
Bagging	parallel	parallel
Boosting	sequential	parallel

Applicability



Now boostings are **used in production a lot**.

They shine bright on **tabular data** giving relatively light (in both training and inference) and accurate algorithm.
Neural networks barely can outperform GB in this case, and even in this case their costs become a problem (still people try to make breakthrough).

For so called unstructured data (images, texts, etc) it works not so well because of trees nature to treat each feature independently.

Summary



1. Gradient Boosting is a powerful meta-algorithm
2. Basic models are trained sequentially
3. Each basic model predicts gradient of loss function of previous ensemble
    a. residuals of previous ensemble - only for MSE!
4. Basic model is always decision tree
5. Inference is done in parallel
6. GB is best for tabular data

Literature



1. https://en.wikipedia.org/wiki/Jerome_H._Friedman
2. Great course on boostings!
    a. https://apxml.com/courses/mastering-gradient-boosting-algorithms
3.

GB derivation for MSE



$$L \in R; y \in R^n; p \in R^n$$

$$

\textstyle{\frac{\partial}{\partial{p}}} L(y, p) = [{\frac{\partial}{\partial{p_1}}} L(y, p), {\frac{\partial}{\partial{p_2}}} L(y, p), ...]

$$

$$

1/n \cdot \textstyle{\frac{\partial}{\partial{p}}} \sum_i (y_i - p_i)^2 = 1/n [-2 (y_1 - p_1), -2 (y_2 - p_2), ...]

$$

Gradient Boosting examples

girafe ai

03

Gradient boosting



What we need:

- Data
- Loss function and its gradient
- Family of algorithms (with constraints if necessary)
- Number of iterations M
- Initial value (GBM by Friedman): constant

Gradient boosting: example

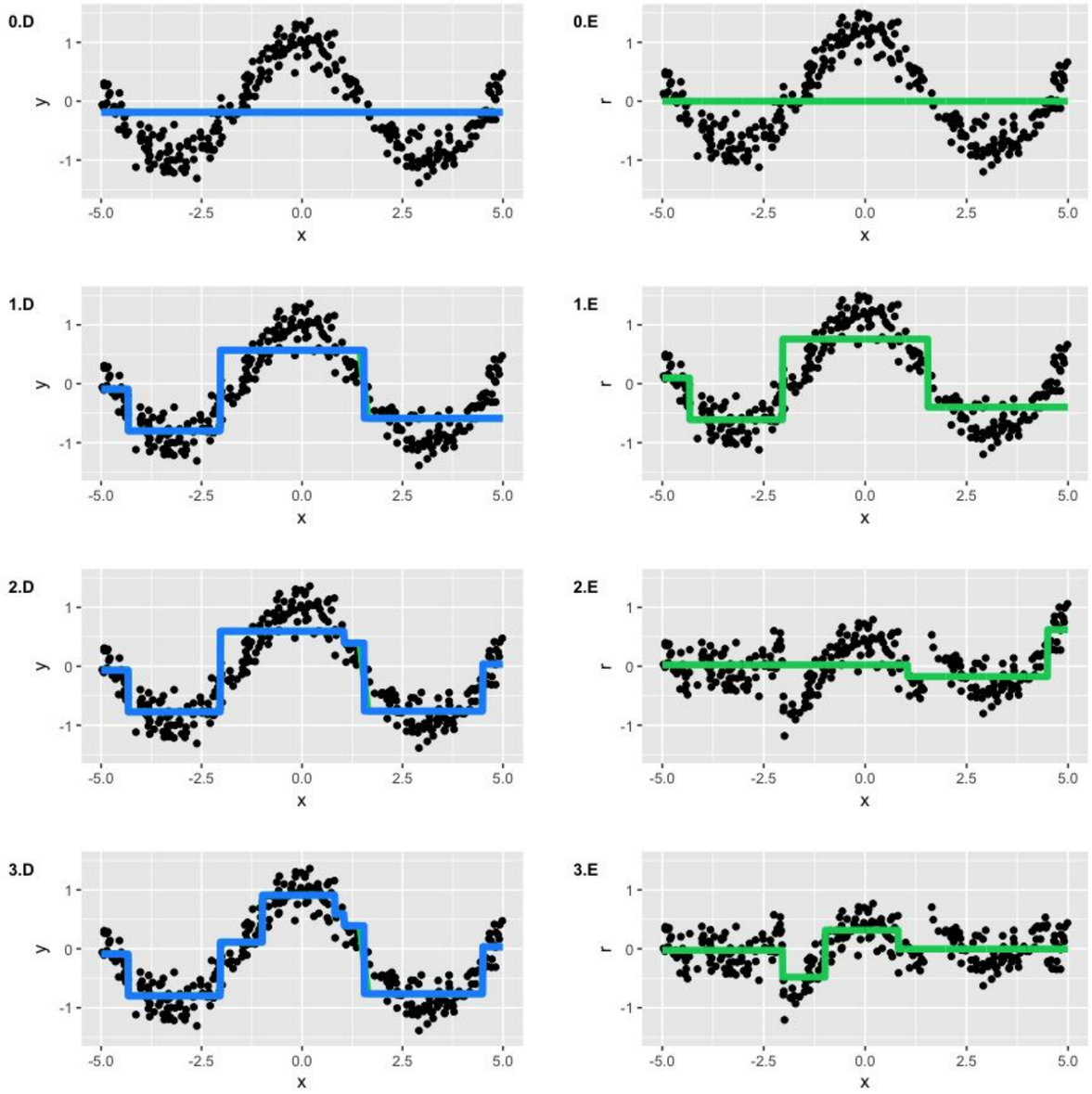


What we need:

- Data: toy dataset $y = cos(x) + \epsilon, \epsilon \sim \mathcal{N}(0, \frac{1}{5}), x \in [-5, 5]$
- Loss function: MSE
- Family of algorithms: decision trees with depth 2
- Number of iterations M = 3
- Initial value: just mean value

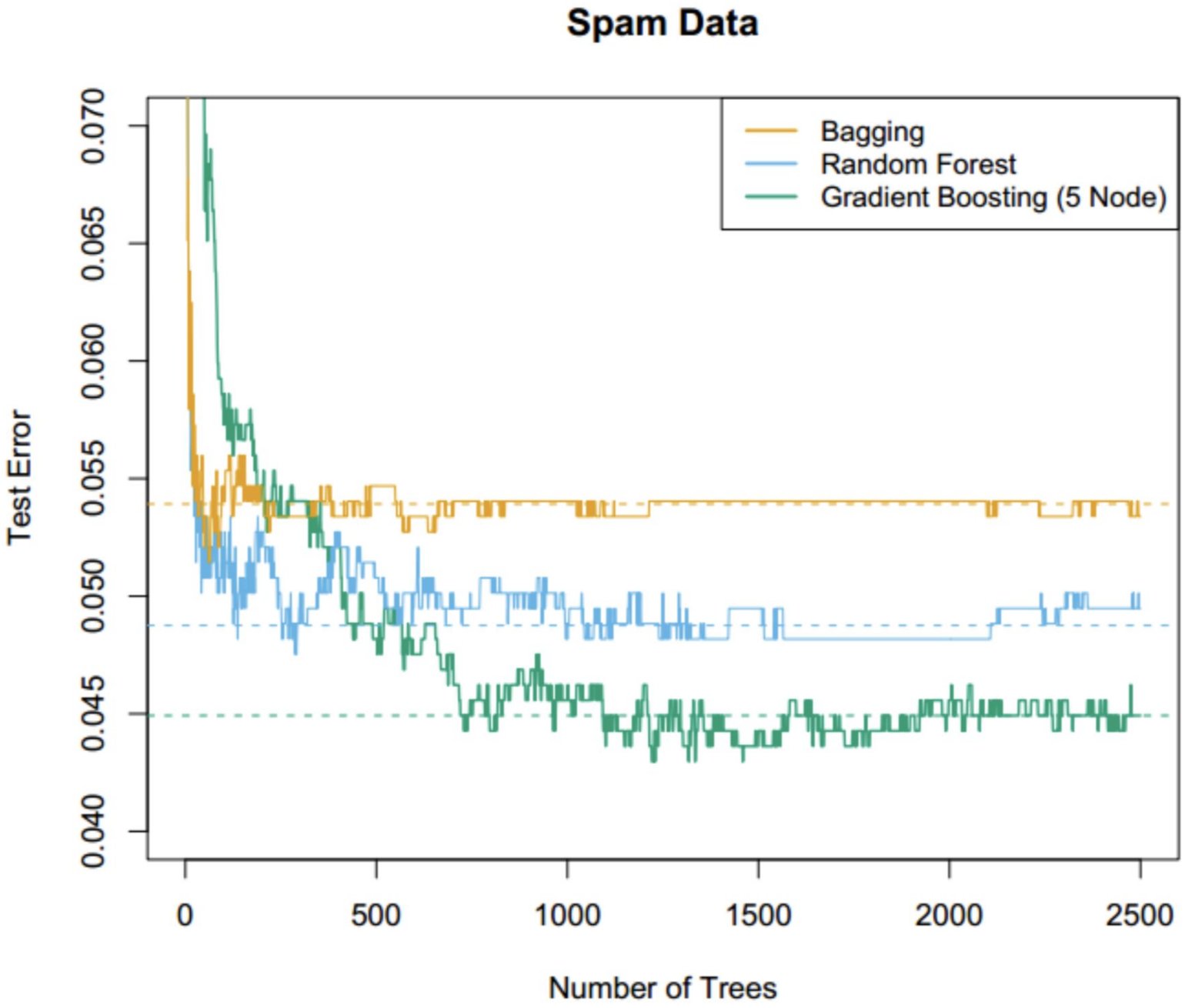
https://habr.com/ru/company/ods/blog/327250/

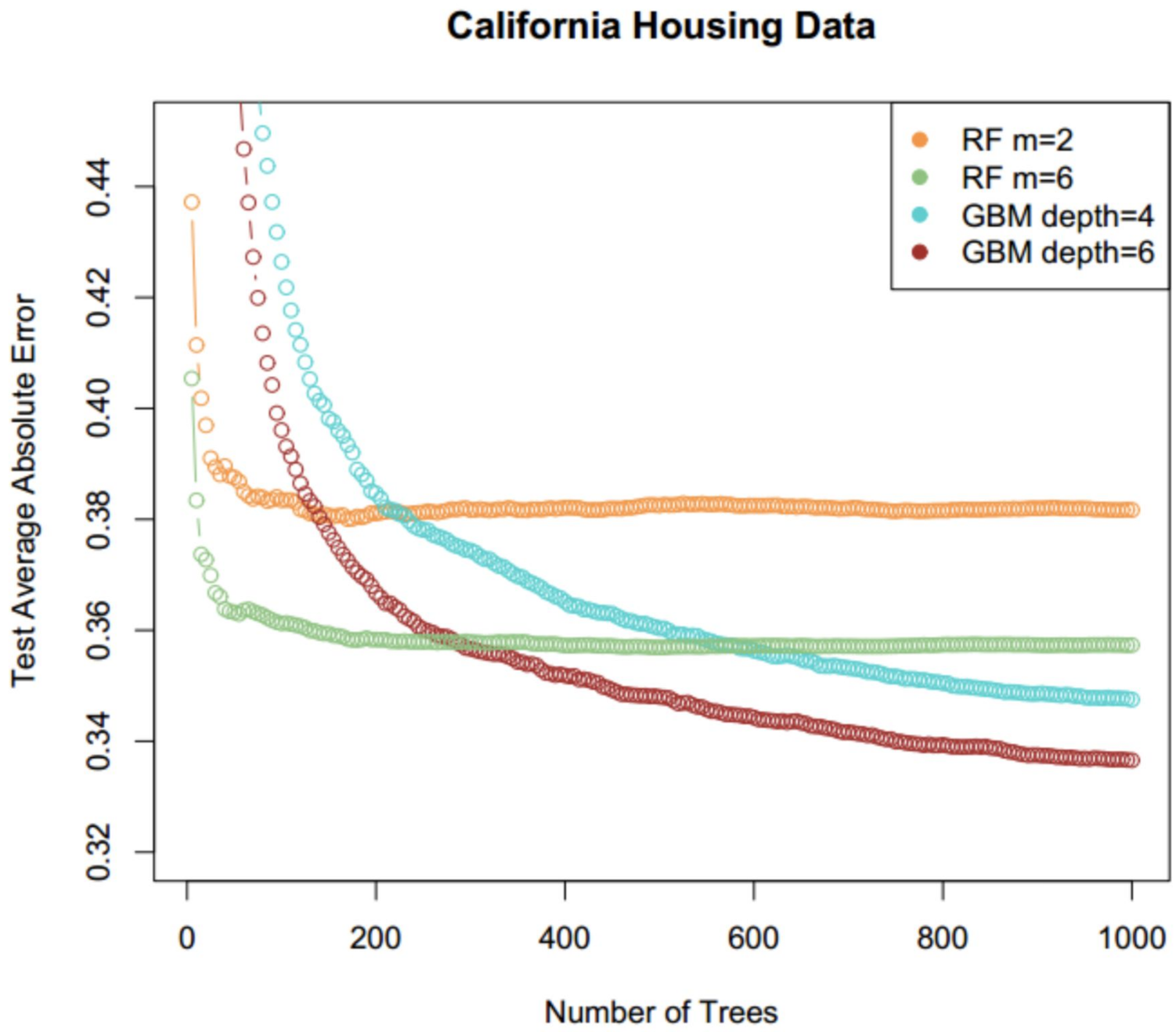
Gradient boosting: example



Left: full ensemble on each step.

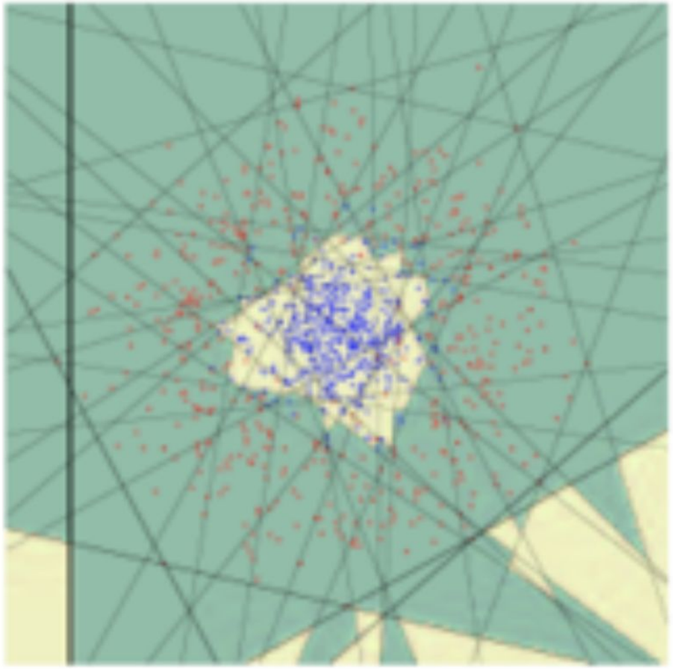
Right: additional tree decisions

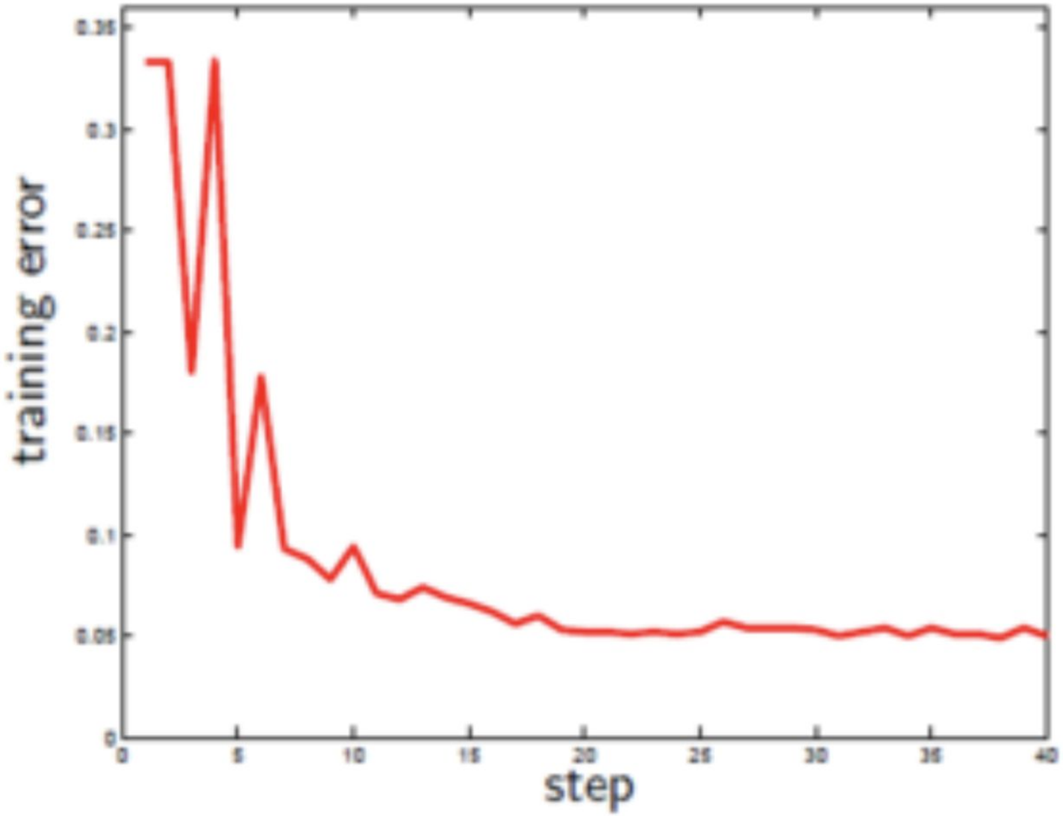

Spam Data
Bagging
Random Forest
Gradient Boosting (5 Node)
Test Error
0.040
0.045
0.050
0.055
0.060
0.065
0.070
0
500
1000
1500
2000
2500
Number of Trees


California Housing Data
Test Average Absolute Error
0.32
0.34
0.36
0.38
0.40
0.42
0.44
0
200
400
600
800
1000
Number of Trees
RF m=2
RF m=6
GBM depth=4
GBM depth=6

Boosting with linear classification methods

$t = 40$





Libraries for Gradient Boosting

girafe ai

04

XGboost by community



- The first leader to arrive!
- Started being developed in 2014
- Best documentation compared to others
- Solves all main tasks: regression, classification, ranking
- Quality is similar to all others
- Nowadays works 20-30% slower than others
- Supports GPU train and inference

https://xgboost.readthedocs.io/en/stable/

LightGBM



- Also used to be internal instrument at Microsoft
- Supports

https://lightgbm.readthedocs.io/en/latest/index.html

Catboost



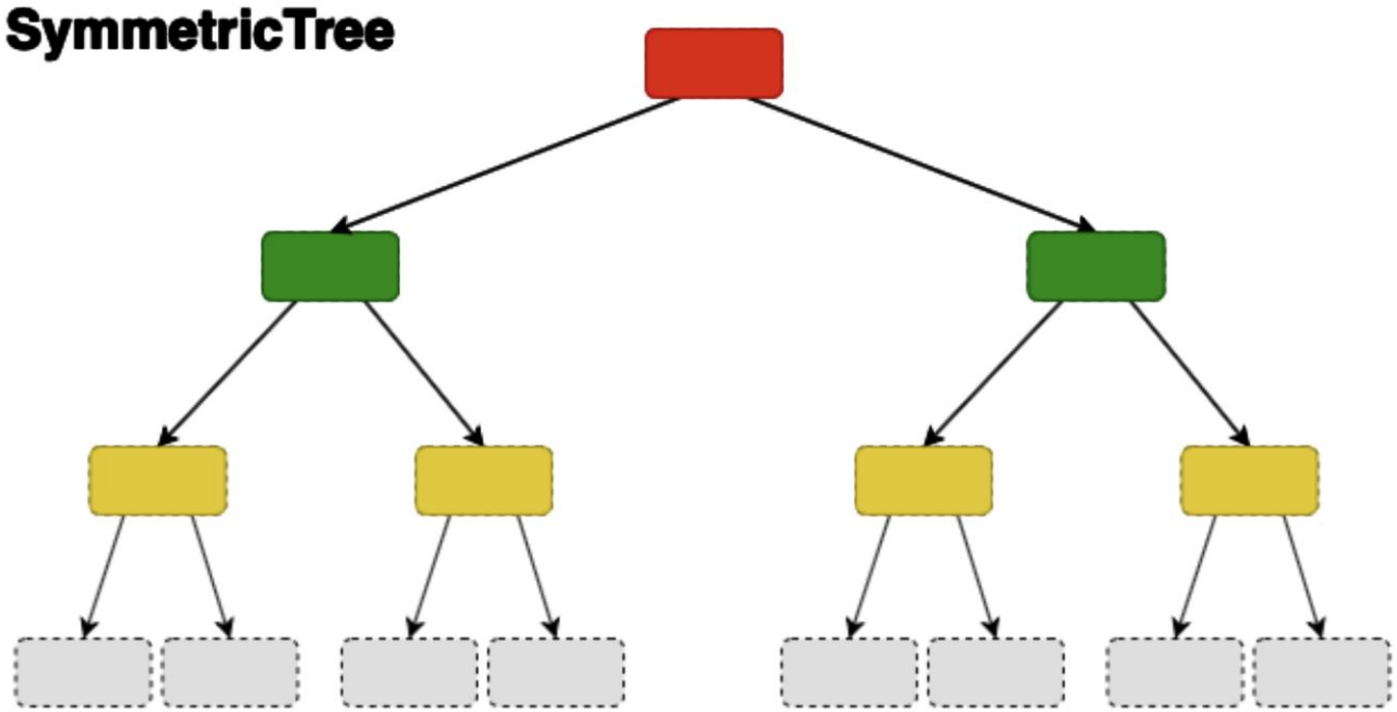
Made by Yandex
https://catboost.ai/

- Used to be internal instrument “Matrixnet”
- Solves all main tasks: regression, classification, ranking
- Tailored to process categorical features out of the box
- Supports GPU train and inference
- Inference bindings for most web languages

Dive to Catboost

- Explained by core developer for girafe-ai, slides
- https://apxml.com/courses/mastering-gradient-boosting-algorithms/chapter-6-catboost-gradient-boosting/catboost-oblivious-trees

Форма дерева – CatBoost – SymmetricTree



Main contemporary instruments

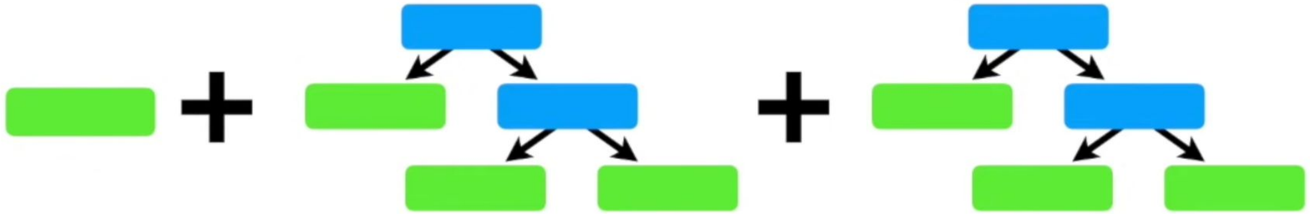
1. Catboost by Yandex
   a. Great default
2. LightGBM by Microsoft
   a. another good choice
3. XGboost by community
   a. Great documentation, poor speed

Definitely not sklearn!

Boosting explained in verse!

1. Boosting explained
2. XGBoost expained

Gradient Boost Part 1...

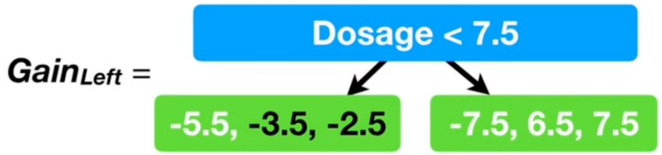


...Regression Main Ideas!!!



Predicted Drug Effectiveness

0.5



Dosage	Drug Effectiveness	Residuals
5	-5	-5.5
10	-7	-7.5
21	7	6.5
25	8	7.5

Dosage	Drug Effectiveness	Residuals
???	-3	-3.5
???	-2	-2.5

The first **Gain** value, which we will call ***Gain*$_{Left}$**, is calculated by putting all of the **Residuals** with missing **Dosage** values into the leaf on the left.

More on boosting



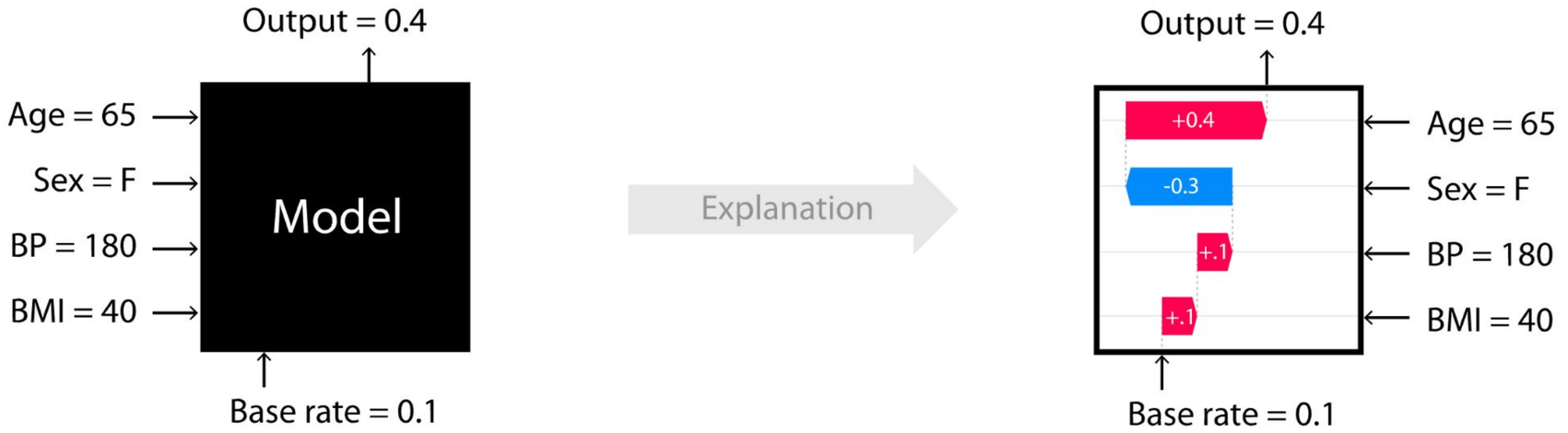
- https://habr.com/ru/companies/ods/articles/645887/
- https://neptune.ai/blog/when-to-choose-catboost-over-xgboost-or-lightgbm
- https://towardsdatascience.com/catboost-vs-lightgbm-vs-xgboost-c80f40662924
- https://www.springboard.com/blog/data-science/xgboost-random-forest-catboost-lightgbm/
- https://towardsdatascience.com/performance-comparison-catboost-vs-xgboost-and-catboost-vs-lightgbm-886c1c96db64

SHAP values

girafe ai

05

Shap values



https://github.com/shap/shap

Shap values calculation

Coalition values

$C_{12} = 10{,}000$
$C_1 = 7{,}500$
$C_2 = 5{,}000$
$C_0 = 0$

Marginal contribution

The increase in a coalition's value due to a player joining that coalition



$C_{12} - C_2 = 5{,}000$
$C_1 - C_0 = 7{,}500$

$(5{,}000+7{,}500)/2$
$= \mathbf{\$6{,}250}$



$C_{12} - C_1 = 2{,}500$
$C_2 - C_0 = 5{,}000$

$(2500+5000)/2$
$= \mathbf{\$3{,}750}$

Shap values calculation



$C_{123} = 10{,}000$
$C_{0} = 0$

$C_{12} = 7{,}500$
$C_{13} = 7{,}500$
$C_{23} = 5{,}000$

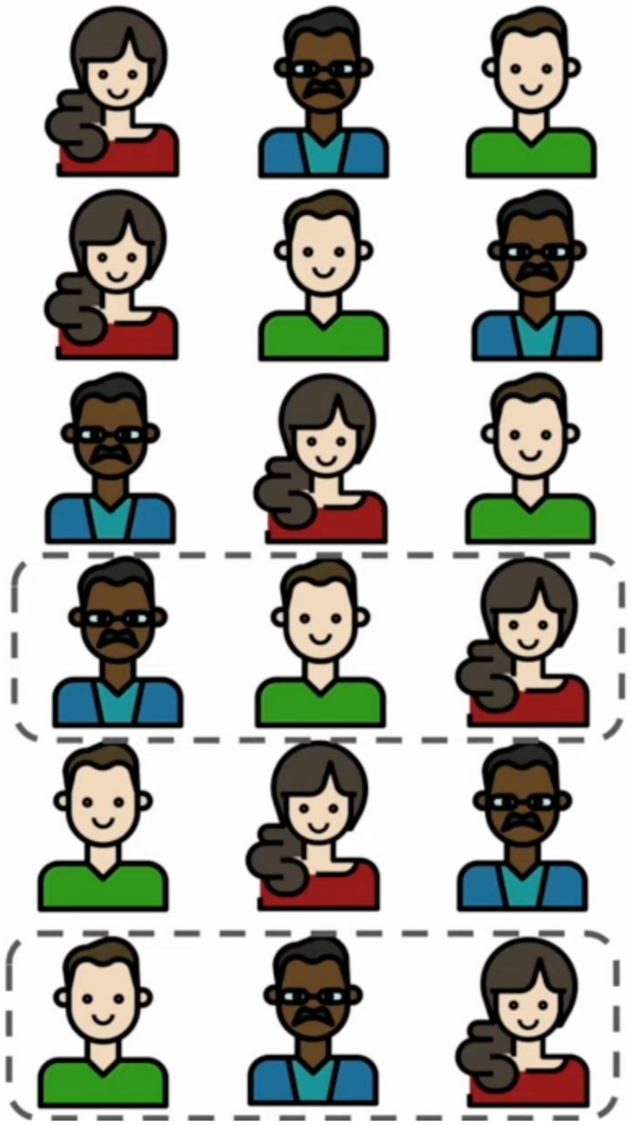
$C_{1} = 5{,}000$
$C_{2} = 5{,}000$
$C_{3} = 0$



$C_{123} - C_{23} = 5{,}000$
$C_{12} - C_{2} = 2{,}500$
$C_{13} - C_{3} = 7{,}500$
$C_{1} - C_{0} = 5{,}000$

$5{,}000*(⅓) + 2{,}500*(⅙)$
$+ 7{,}500*(⅙) + 5{,}000*(⅓)$
$=$ **$5,000**

Shap values calculation



$C_{123} - C_{23} = 5{,}000$

$P(C_{123} - C_{23})$ = probability that player 1 makes a marginal contribution to a coalition of player 2 and 3

3! = 6

2! * 1! = 2

$P(C_{123} - C_{23}) = 2/6 = 1/3$

Shap values axioms

Efficiency



Symmetry



Null player



Additivity



Shap values

$$\phi_i = \sum_{S \subseteq \{1,\ldots,p\}\{i\}} \underbrace{\frac{|S|!(p-|S|-1)!}{p!}}_{\text{Weight}} \underbrace{[val(S \cup \{i\}) - val(S)]}_{\text{Marginal contribution of player i to coalition S}}$$

p! = number of ways to form a coalition of p players

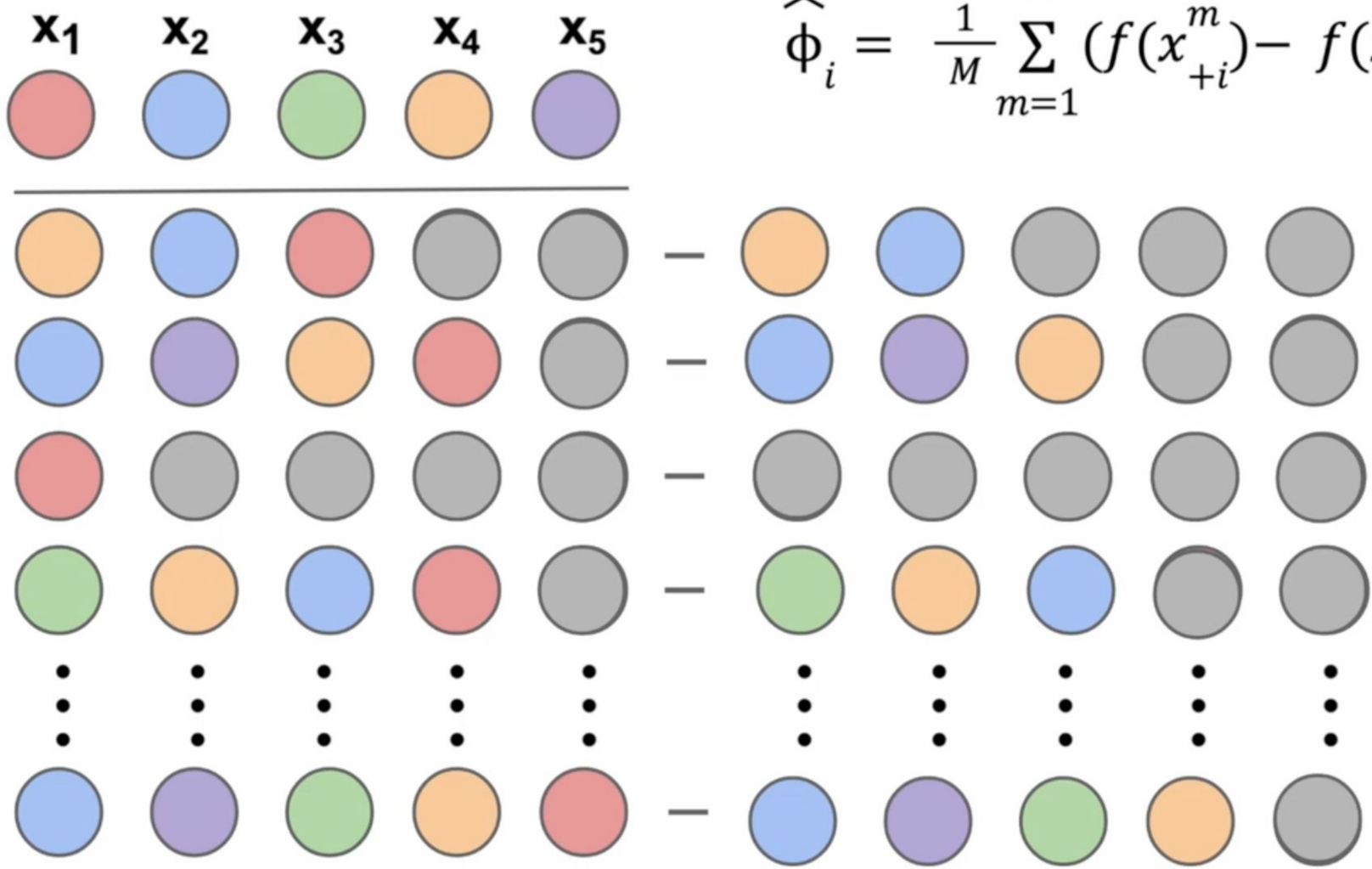
|S| = number of players in coalition S

|S|! = number of ways coalition S can form

(p-|S|-1)! = number of ways players can join after player i joins

https://youtu.be/UJeu29wq7d0

Shap values for ML



$$\widehat{\phi}_i = \frac{1}{M}\sum_{m=1}^{M}\left(f(x_{+i}^{m}) - f(x_{-i}^{m})\right)$$

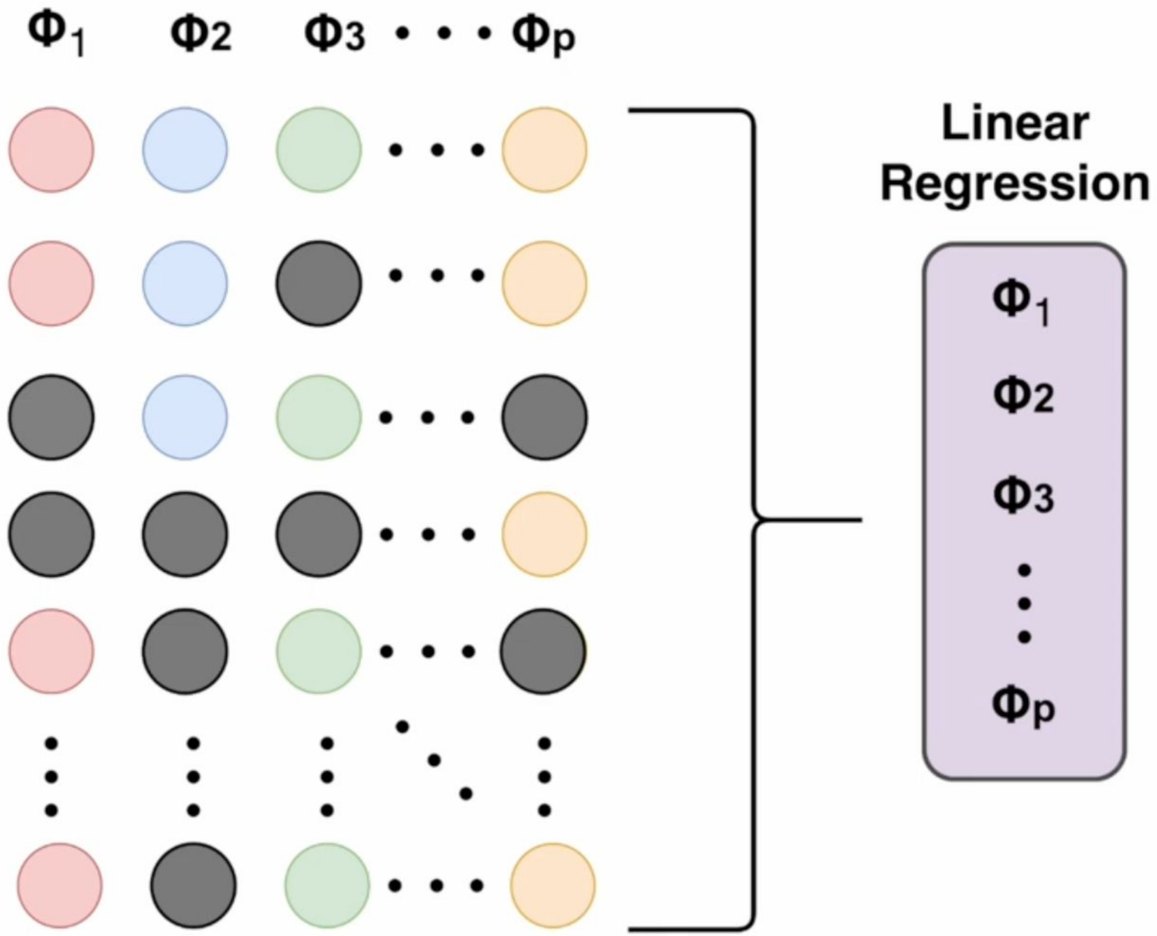
Shap values for ML

KernelSHAP



TreeSHAP





Libraries for shap values



- https://shap.readthedocs.io/en/latest/index.html
  - https://github.com/shap/shap
- https://github.com/mmschlk/shapiq
-

Materials on shap values



- shap values introduction https://www.youtube.com/watch?v=MQ6fFDwjuco&list=PLqDyyww9y-1SJqMw92x90qPYpHgahDLIK
- detailed shapley values https://www.youtube.com/watch?v=4RkhsIz14Yc
- Explainable AI
  - https://www.youtube.com/watch?v=VY7SCl_DFho&list=PLqDyyww9y-1SwNZ-6CmvfXDAOdLS7yUQ4
  - https://www.youtube.com/watch?v=OZJ1IgSgP9E&list=PLV8yxwGOxvvovp-j6ztxhF3QcKXT6vORU
- [ru] https://t.me/smalldatascience/1019
  - https://vkvideo.ru/video-232735712_456239019

Hyperparameter optimization

girafe ai

06

Optimization note

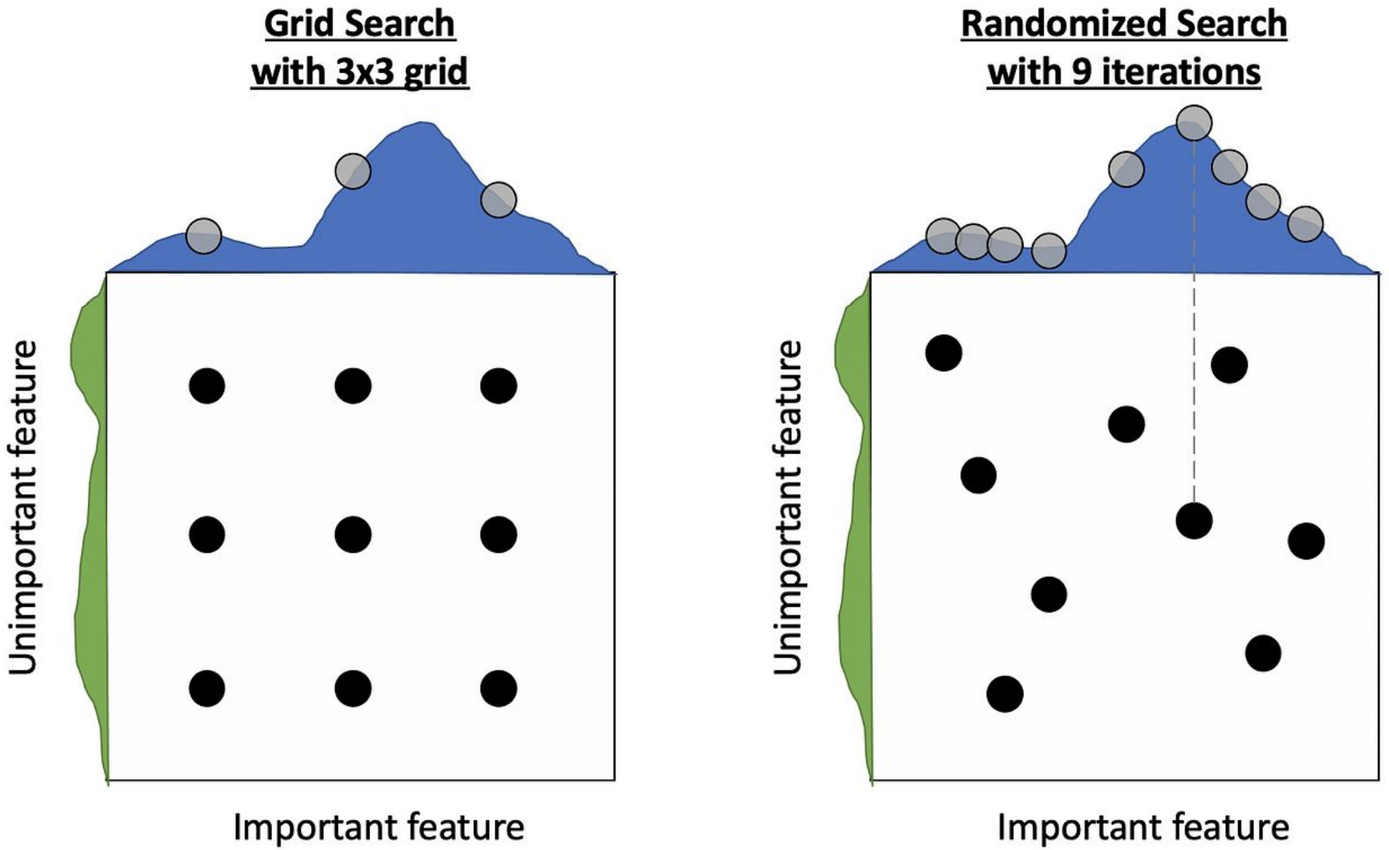


In optimization theory methods are associated with the order of derivatives they use. Main ones are:

- first order optimization (gradient based optimization)
  - use gradient of optimized function
  - e.g. SGD which we discussed
- second order optimization (Newton's method)
  - use Hessian matrix
  - they are quite slow
- zero order (black box optimization)
  - don't need gradient, only values of optimized function
  - that's what we are interested in today

0-order optimization approaches

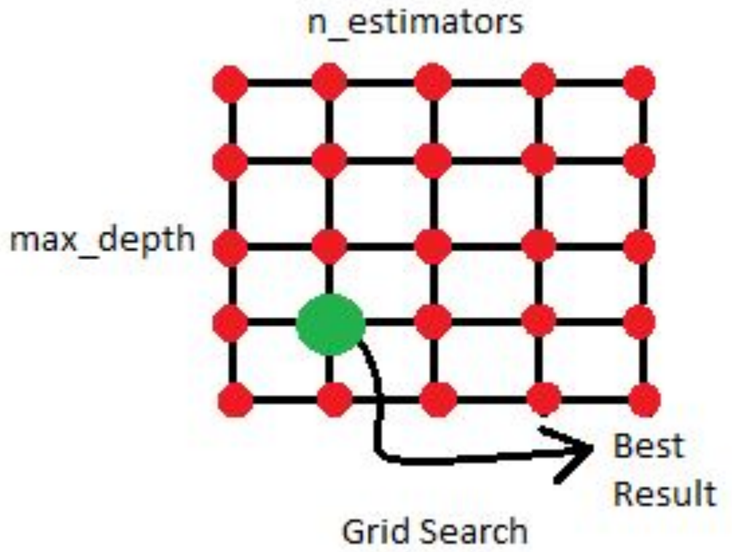
1. Manual trials
2. Grid search
3. Random search

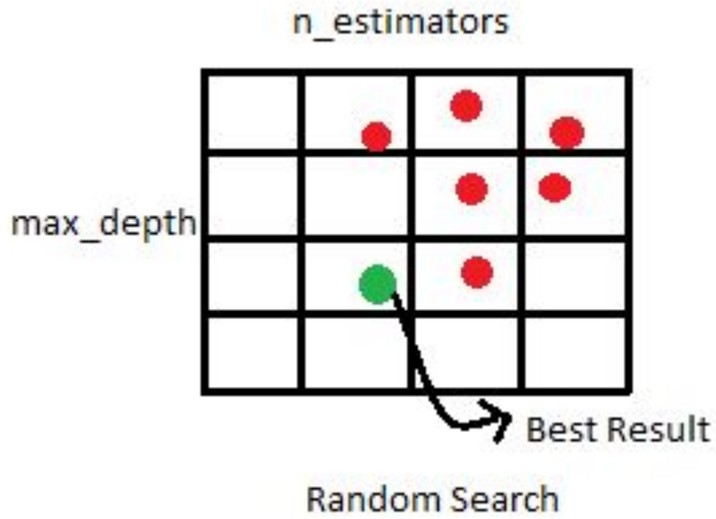


In theory

0-order optimization approaches

1. Manual trials
2. Grid search
3. Random search

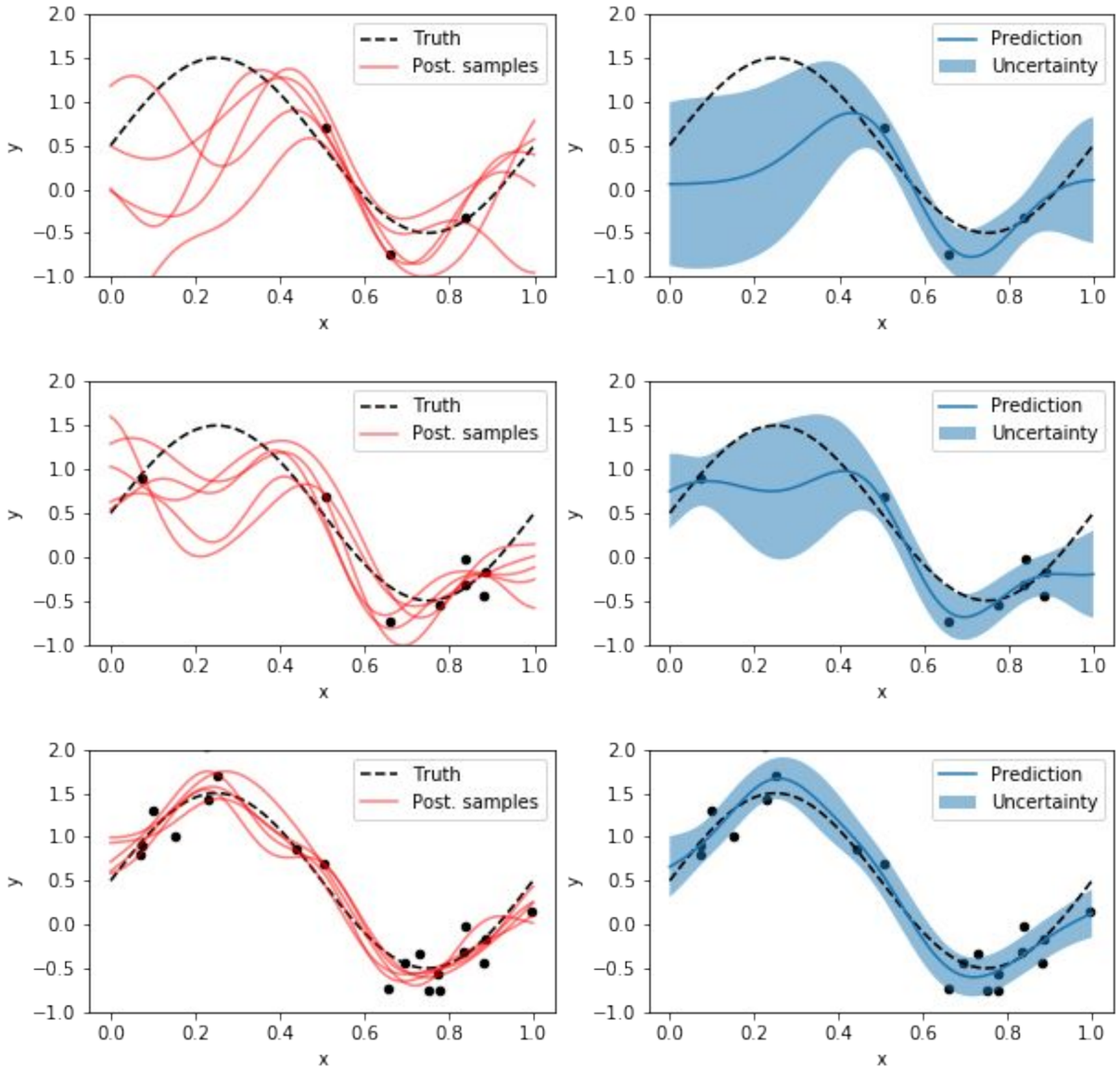




In practice

0-order optimization approaches

1. Manual trials
2. Grid search
3. Random search
4. Bayesian methods
5. Evolutionary methods



Main libraries

- Hyperopt
- Optuna
  - Documentation
  - Samplers list





OPTUNA

Revise

Lecture 5:
Decision trees
Bagging



1. Bias-Variance-Noise decomposition
    a. Bias-Variance tradeoff
    b. Beyond MSE
    c. Double descent
2. Gradient boosting
    a. Intuition
    b. Boosting as gradient descent
    c. Basic model choice
    d. Parallelism
3. Libraries
    a. XGboost, LightGBM, Catboost
4. Feature importances
    a. SHAP
5. Hyperparameter optimization
    a. Optuna

Thanks for attention!

Questions?



girafe
ai

